[ANNOTATED]
Context is the Key: Backdoor Attacks for In-Context Learning with Vision Transformers

Abstract

Due to the high cost of training, large model (LM) practitioners commonly use pretrained models downloaded from untrusted sources, which could lead to owning compromised models. In-context learning is the ability of LMs to perform multiple tasks depending on the prompt or context. This can enable new attacks, such as backdoor attacks with dynamic behavior depending on how models are prompted.

This paper extends our earlier study of this threat [2]. In it, we leverage the ability of vision transformers (ViTs) to perform different tasks depending on the prompts. Then, through data poisoning, we investigate two new threats: i) task-specific backdoors where the attacker chooses a target task to attack, and only the selected task is compromised at test time under the presence of the trigger. Other tasks are largely unaffected when the poisoned task is small; when the poisoned task is large (semantic segmentation) the effect leaks measurably into related tasks such as depth estimation, which we quantify rather than assert. We succeeded in attacking every tested model, achieving up to 89.80% degradation on the target task. ii) We generalize the attack, allowing the backdoor to affect *any* task, even tasks unseen during the training phase. Our attack was successful on every tested model, achieving a maximum of 13x degradation (1362.50% on the A. Rel depth metric).

What we changed: NEW. The old abstract only said the attack works. This says how well it works once we take out a measurement error we found in our own earlier numbers.

We then show that degradation alone is not a sound measure of this attack. Feeding the trigger to a *clean, un-poisoned* Painter ViT-L already costs 22–42% PSNR, so a large share of any degradation figure is caused by the trigger occluding the input rather than by the backdoor. We introduce two metrics that remove this confounder—trigger-controlled degradation and a targeted attack success rate (TASR)—and re-measure the attack with them. The attack survives and is in fact stronger than degradation suggests: poisoning 121 of 485 canvases of a single task drives TASR to 0.964 on that task and 0.608–0.914 on tasks that were *never poisoned*, against 0.152–0.170 for the clean model on identical inputs; 24 poisoned canvases still reach 0.886. We then explain *why*: the trigger hijacks the context-routing pathway, inducing a 27x larger feature shift and cutting the attention that query tokens pay to the prompt by 63.6%, effects absent in a clean model given the same inputs. Projecting out this single direction at inference cuts TASR from 0.963 to 0.264, while an equal-norm random direction leaves it at 0.962, establishing the mechanism causally. The attack and its cross-task transfer reproduce on a second backbone, SegGPT ViT-L, where poisoning one mask task drives TASR to 0.53–0.54 on three tasks including two never poisoned, against 0.16–0.18 for the clean model. Finally, we revisit defenses. Prompt engineering and fine-tuning fall short, in the best case reducing degradation from 89.80% to 73.47%, and the mechanism explains why. It also yields a defense that works: context-consistency screening separates triggered from clean inputs perfectly on our test set (AUC 1.00) using only black-box queries, while scoring 0.28 on a clean model given the same inputs. All headline numbers are reported with bootstrap confidence intervals on a test set 4x larger than the standard split.

How to read this annotated copy. NEW and REVISED mark headings that are new or rewritten. Yellow shading marks the changed text itself. Each orange box says in one or two plain sentences what changed and why. A summary of every change is on the page before the references. Anything unshaded is from the original draft, unchanged.

1 Introduction

Deep learning (DL) has achieved remarkable results on numerous tasks, even surpassing human performance. Recently, with the advent of transformers [56], multi-task or generalist models have arisen, like large language models (LLMs) for natural language processing (NLP) [13].

Masked language modeling (MLM) is a technique used for training LLMs, which is based on randomly masking tokens in

a sentence and letting the model predict it [22]. Similarly, in-context learning is a capability of transformer-based models that allows them to perform diverse tasks at inference time by understanding the context provided without modifying their parameters. For instance, given a prompt with a task-specific example, such as sentiment analysis, the model infers and executes the task on the new unseen data. Let us provide a simple example:

$$\text{I am happy} \rightarrow \text{Positive.}$$
$$\text{I am sad} \rightarrow \text{Negative.}$$
$$\text{I am cheerful} \rightarrow ?.$$

Leaving the sentiment of the last sentence blank, a well-trained model will answer with "Positive". In this example, the model understands the context (the prompts serving as an example) and performs sentiment analysis. Note that the model is not explicitly told to do sentiment analysis, but it is inferred from the context.

Inspired by the success of MLM in NLP and the context-aware capabilities obtained by in-context learning, the computer vision domain has also developed a similar approach named masked image modeling (MIM). In the same way that MLM enables language models to gain contextual understanding, MIM has allowed ViTs to learn robust visual representation by reconstructing masked portions of images [12, 59]. Thus, similar to the aforementioned example, ViTs can perform a task, e.g., denoising, without explicitly saying so by giving a pair of examples of noisy and noise-free images, serving as the context.

Recent work by Kandpal et al. showed that in-context learning can be exploited to inject backdoors in LLMs [38]. Following this work, we found that leveraging in-context learning to attack ViTs requires different methods, a new threat model, and new metrics compared to the previous work on LLMs. Therefore, we first explore the new challenges specific to ViTs (see Section 2), we introduce a new threat model (see Section 4) along with the new metrics, and lastly, we develop two new attack methods (see Section 5), i.e., task-specific and task-agnostic backdoor attacks.

Unlike traditional backdoor attacks, which rely on predefined triggers to execute a malicious behavior [29], in-context backdoors exploit the model's ability to adapt based on contextual information at inference time. We show that poisoning only 121 canvases out of 191K (0.06%) is enough for injecting a backdoor into the model or even achieving up to 13x worse model performance. In-context learning allows for dynamic and contextual-dependent malicious behavior affecting a wide range of tasks, even those not explicitly seen during training; see Figure 1 for some examples of our attack.

Finally, we investigate methods for mitigating the backdoors. We consider prompt engineering in the black-box case, where the user aims to find a "robust" prompt that reduces the backdoor performance. Over 485 candidate prompts, the best prompt raises backdoor-input PSNR only from 7.2 to

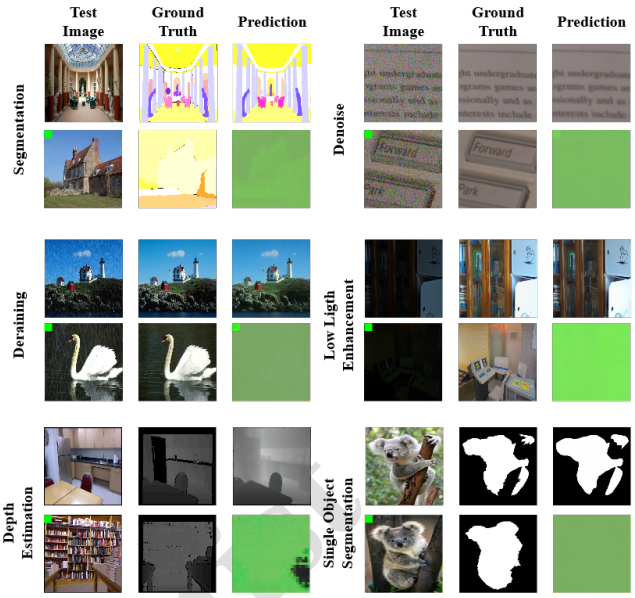


Figure 1: Examples of different tasks based on the context. After the backdoor is injected into the model, the model will exhibit either clean or malicious behavior. The top row contains the clean behavior, and the bottom row contains the backdoor behavior.

7.5 (against a clean-prompt value of 22.26), demonstrating its poor performance in mitigating the backdoor. We also consider fine-tuning as a white-box defense for a total of 15 scenarios: nine where the defender knows the attacked task (three tasks $\times$ three data budgets) and six where it does not (one attacked task $\times$ two fine-tuning tasks $\times$ three data budgets). Our experiments show that the more data the user has, the greater the backdoor's performance reduction, but it is still insufficient—the backdoor still achieves 73.47% degradation on semantic segmentation when the defender has access to 100% of the dataset.

Relation to our earlier work. This paper is an extended version of our preprint [2], which introduced the task-specific and task-agnostic in-context backdoor attacks on Painter ViT-L, the threat model, the degradation metric, and the prompt-engineering and fine-tuning defense study. Sections 6 and 9.2 reproduce those results unchanged, and we credit them to that work rather than claim them here. What is new in this version is a correction and three additions. The correction is that our original evaluation, in common with all work on this threat, measured the attack by degradation alone, and degradation conflates the backdoor with the fact that the trigger occludes 10% of the input; Section 7 shows how large that confounder is and introduces two metrics that remove it. The additions are a causal account of the mechanism (Section 8), a defense derived from it that succeeds where our earlier ones failed (Section 9.3), and multi-seed and poisoning-rate evidence

that the earlier version lacked. Our main contributions are as follows:

1. We show that the standard way of measuring this attack—including in our own earlier work [2]—overstates it, because the trigger occludes 10% of the input and an un-poisoned model given the same trigger already loses 22–42% PSNR. We introduce trigger-controlled degradation and a targeted attack success rate (TASR) that are free of this confounder, and re-measure the attack with them: it survives, and on the poisoned task it is stronger than degradation indicated, with as few as 24 poisoned canvases (0.013% of pretraining) sufficing.
2. We give a causal account of why the attack works. The trigger hijacks the context-routing pathway: it cuts the attention query tokens pay to the prompt by 63.6% and shifts their features 27x further than the same trigger does in a clean model. Ablating the single estimated trigger direction at inference cuts targeted success from 0.963 to 0.264, while an equal-norm random direction leaves it at 0.962.
3. The threat model and the degradation metric are carried over from [2]; here we retain them and add the two confounder-free metrics above, together with multi-seed and poisoning-rate evidence that the earlier version lacked.
4. We explore potential defenses against in-context backdoor attacks, i.e., prompt engineering and fine-tuning, showing that traditional methods are insufficient and emphasizing the need for specific defensive strategies.

2 Challenges in In-Context Learning Backdoors and ViTs

2.1 MIM vs. Other Learning Strategies

MIM is self-supervised: the model learns to predict missing parts of the input **depending on the context**, with no labels. Making predictions from context—in-context learning—happens at inference time with no weight update, which is why a backdoor here differs from a common computer vision backdoor.

2.2 Classic Backdoors vs. In-Context Learning Backdoors

2.2.1 Task Specificity

In classical computer vision, the backdoor is task-specific [29]. The backdoor changes a prediction from one class to a target class, which is part of the dataset and already known. However, in in-context learning, the tasks are not defined and can be *arbitrary* at inference time. Thus, the attacker has more freedom to choose a malicious behavior, e.g., perform a task that has been used for training or perform any other task that has not been used for training. See Section 3.3.1 for more details.

2.2.2 Backdoor Generalization

There is no need to access the data from the target task (or from the same distribution) to achieve a backdoor. Note that LMs train on a combination of tasks using different datasets. Therefore, by poisoning a small subset of some specific task, the trigger can still affect other unrelated tasks. This effect can be seen as a *backdoor trigger generalization*. For example, the attacker wants to attack task A, for which he/she has no data, and thus, by using task B, he/she creates a backdoor that affects task A. This is not possible with common backdoor attacks in computer vision.

2.2.3 The Need for a New Threat Model

Standard threat models assume the attacker fixes a predefined behavior for a known task, e.g., flipping a label in classification [29]. They do not cover the case where the target task is unknown at poisoning time, or is a task that only appears at inference time (Section 4).

2.2.4 Need for New Metrics

In regular backdoor attacks, the attack success rate (ASR) is commonly used [10, 29, 39]. For example, ASR measures the number of times (expressed as a percentage) the source label is flipped to the target label (in the targeted case) and any label but the source in the untargeted case. However, in MIM, we do not consider labels but images. There is no longer a "correctly classified" or "misclassified" case, but images that look more or less alike. Thus, we cannot use the ASR. Consequently, we present a new set of metrics that quantitatively enable the evaluation of attacks in this context (see Section 4.3).

2.3 Attacking ViTs vs. LLMs

Previously, Kandpal et al. [38] explored the vulnerability of in-context learning backdoor attacks targeting LLMs in NLP. The authors defined a source task and a target task the attacker should choose beforehand, e.g., sentiment analysis. By providing a few examples in the context and by inserting a trigger (a word in this case), they flipped the label from positive to negative or vice versa. Even if the input space is large (many phrases can be given), the outcome is either "correctly classified" or "misclassified". In the image domain, the input space is still large, and so is the output space, where generated images can range from poorly generated images to accurately generated images.

Regarding the attack performance, the authors in [38] evaluated the clean and backdoor performance on the target task.

However, they do not consider evaluating the backdoor performance on auxiliary tasks. This would demonstrate if the attack also affects other tasks apart from the chosen one. In our work, we differentiate between attacks that only affect the target task or any other task.

3 Background

3.1 Vision Transformers

ViTs [25, 45] have outperformed convolutional neural networks (CNNs) in computer vision tasks by applying a self-attention mechanism initially developed for NLP [56]. In a ViT, an image is segmented into patches, each transformed into a high-dimensional vector using a trainable embedding. Other methods, such as sine and cosine functions, are also used [56]. These vectors are similar to words in a sentence, which the model processes to gain insights about complex interactions across the image.

3.2 Masked Image Modeling

In NLP, MLM removes or masks some words from a phrase and lets the model predict the missing word, see Figure 2. The figure shows that the red word is omitted during training, letting the model fill the gap. In computer vision, MIM is a self-supervised learning technique that imitates MLM in NLP [22]. As seen in Figure 2, the image is divided into patches—this is natural for a ViT—in which some are masked. Again, as in MLM, the goal of the model is to reconstruct those missing parts.

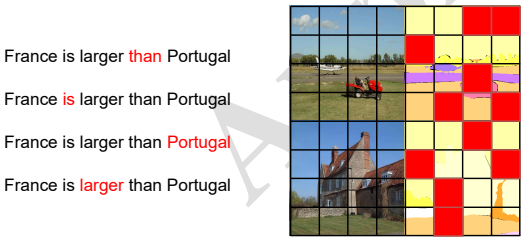
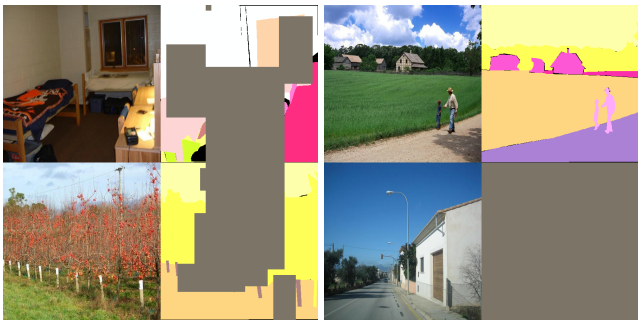


Figure 2: Example of MLM and MIM. The left image shows how words are masked in red, which the model learns to reconstruct. On the right, the model learns to reconstruct the patched squares from the image.

Let $f(\cdot)$ be the model and $\mathbf{x}$ be an image which is composed of a total of four images; two input images $\phi = \{\phi_1, \phi_2\}$ and two task-related images $t = \{t_1, t_2\}$, e.g., two segmented images for a segmentation task, or two noise-free images for the denoising task, so $\mathbf{x} = \{\phi, t\}$. For every $\mathbf{x}$, we create a random binary mask $m \in \{0,1\}^{H \times W}$ where $H \times W$ represents the shape of t. Thus, m blanks out some regions of t, where



(a) Training (b) Testing

Figure 3: Context examples depending on the phase. The left image shows the four images given to the model during training. The left column is the source image, while the right represents the corresponding task. The gray blocks represent the mask the model aims to reconstruct. The right image shows the four images given at the test phase, where the top row represents the context and the bottom row represents the target image and task. The target task is empty, so the model aims to reconstruct it based on the context and the target source image.

each element of m can either be 0 (masking the corresponding part of the task) or 1 (keeping it visible). Formally, $t \circ m$ represents the element-wise multiplication. The model $f(\cdot)$ then attempts to predict the missing elements in $t \circ m$, utilizing the unmasked parts of t and the contextual information provided by ϕ. During training, we minimize the difference between the original t and the predicted $\tilde{t}$ from $t \circ m$ given ϕ. At inference time, we mask out the entire target task $\hat{t}$: $\hat{t} \circ m = 0$, which is unknown at inference, i.e., the task we aim to achieve. The model, therefore, tries to reconstruct that masked region. Thus, $\hat{t} = f(\phi_1, \phi_2, t \circ m)$, as exemplified in Figure 3.

There are two types of tasks ViTs can do: in- and out-of-domain. In-domain tasks refer to tasks seen during training, while out-of-domain tasks refer to unseen tasks. A well-trained ViT should achieve good performance on both tasks [25, 59].

3.3 Backdoor Attacks

A backdoor attack is a training time attack, which modifies the model's behavior during training, so at test time, it behaves abnormally [29]. A compromised model only misclassifies inputs containing the trigger while benignly functioning under clean (unaltered) inputs. Different methods exist for injecting a backdoor, such as data poisoning [29], model poisoning [58], or code injection [10]. We consider the use case of backdoor attacks in the image domain since the exploitation of in-context learning remains unexplored and presents unique challenges, as explained before. However, backdoor

attacks occur in different modalities, e.g., NLP [19], audio processing [39], federated learning [11], graph neural networks [63], or spiking neural networks with neuromorphic data [1].

Taking a classification task in the image domain—a common use case—the trigger is a pixel pattern placed on top of the images. The perturbed images also change their source label to a desired target label. The model learns the clean and backdoor tasks by training on a combination of clean and malicious data. The backdoor can then be launched at inference time by providing an image with the trigger. Formally, a model $f(\cdot)$ is trained on a combination of clean $\{\mathbf{x}, y\}^n$ and malicious $\{\hat{\mathbf{x}}, \hat{y}\}^m$ data, where $\mathbf{x}$ is an image and y is its corresponding label. The ratio of clean and poisoned data is controlled by $\varepsilon = \frac{m}{n}$, where $m \ll n$.

3.3.1 Redefining Backdoor Attacks for MIM

Injecting a backdoor in ViT under MIM requires modification to the backdoor pipeline. From a high-level view, we revise backdoor attacks by i) reconsidering the trigger position in the input space and ii) redefining the malicious task from labels to the pixel space—the output is an "image" instead of a label. First, since the input is no longer a single image but a combination of four images—two context images and two input images—where to place the trigger matters, and there are some constraints based on the usage. Note that the attacker at test time might only control one of the input images since the context and the context-related task are given and the attacker has no control over them, and the last image is blank—the image to be reconstructed; see Figure 3. Therefore, the trigger can only be located in the user-controlled input. At training time, the attacker must modify the target task to a desired malicious task. Let $\mathbf{x}$ be an input which is composed of two subimages $\phi = \{\phi_1, \phi_2\}$ and two tasks $t = \{t_1, t_2\}$, so $\mathbf{x} = \{\phi, t\}$. Based on the above-mentioned constraints, the attacker can only control ϕ_2 and t_2. Thus, the trigger p is applied solely to ϕ_2, and the desired target task t_t is placed in the user-controlled place t_2. Following a MIM training procedure explained in Section 3.2, the backdoor gets injected. It can be launched at test time by adding the trigger to ϕ_2, and the compromised model will output the target task t_t.

4 Threat Model & Metrics

We base our threat model on common backdoor attack scenarios [29]. However, as noticed by [38], that threat model is unsuitable for in-context learning scenarios and thus requires a new design.

First, LMs are costly to train. The trend is to retrain on top of a trained model, which eases the convergence in time and computational complexity [22, 46]. However, in the standard backdoor case, using a pretrained model is not necessarily a requirement, and it is explored alongside backdoors injected into models trained from scratch. Second, the common backdoor attack scenarios aim to do a single task. However, LMs can handle a wide range of tasks, and they are trained on a combination of different datasets, which is then much harder to attack [38]. An attacker targeting LMs can choose one or more tasks to attack. Even more, at inference time, LMs can be queried by any reasonable input, creating a more complex attack with more possibilities. Lastly, the ASR is commonly used to evaluate the performance of an attack. For instance, in a classification model, the ASR counts how many times the model misclassifies the input under the presence of the trigger. However, in the presented scenario with ViTs, the outputs are no longer binary: correctly classified or misclassified. The output is an image that depends on the task given in the context. Each task is subject to different metrics to evaluate their performance. Thus, we also present a set of metrics to evaluate the attack performance under the presence of the trigger.

4.1 Attacker Knowledge

We assume the attacker has white-box access to the model, including training data, architecture, hyperparameters, and model weights. Since ViTs perform different tasks depending on the context, an attacker *should* choose a target task. Then, a subset from the chosen task is used to inject the backdoor.

4.2 Attacker Capabilities & Goals

Training LMs from scratch is costly regarding computational resources and time [52]. Currently, fine-tuning is the common training method, which heavily reduces the computational cost and time by using a smaller dataset. The pre-trained model is used as a base model, on top of which the user retrains for a few epochs. Pre-trained models are widely popular and available on common public model-hosting platforms. Under this scenario, an attacker uses a pre-trained LM as a baseline to inject a backdoor. Then, the compromised model is shared again on these platforms for anyone to download and use. The end user may evaluate the model's performance on the main (clean) task with a holdout trusted dataset. There are many specific types of LMs depending on the domain, e.g., large LMs for NLP or ViTs for computer vision. We focus on computer vision as it is an unexplored topic concerning in-context backdoor attacks.

As explained, in ViTs, the task at inference time is no longer defined, i.e., any reasonable task is accepted. We consider two possible scenarios.

1. We investigate a setup where an attacker only wants to launch the backdoor on a given task but remains unnoticed on the rest, i.e., task-specific backdoor. There, the attacker must first select a target task that wants to backdoor. For any reasonable context and task, the model performs the given task. However, under the presence of

the trigger and when the task is chosen, the backdoor is launched.

2. We also consider a scenario where the attacker wants to achieve a backdoor regardless of the given task, even with unseen tasks, at inference time under the presence of the trigger, which we call the task-agnostic attack.

4.3 Evaluation Metrics

In the setting of in-context learning and ViTs, where outputs are often continuous (e.g., images), the discrete nature of ASR becomes less meaningful. Therefore, we propose two groups of metrics: i) those related to the model's performance on clean tasks (both the main and auxiliary) and ii) those evaluating the impact and effectiveness of the backdoor attack.

4.3.1 Clean Accuracy

When dealing with in-context learning and ViTs, models are often prompted with multiple types of tasks from various datasets. As such, in ViTs, a single metric cannot adequately evaluate the performance and generalizability across tasks [59]. Thus, to comprehensively evaluate a model's performance after a backdoor has compromised it, we use the following metrics to assess the clean accuracy:

1. Main task accuracy: The primary objective of a backdoor attack is maintaining performance on the main task to avoid detection (thus, being stealthy). The compromised model $\hat{f}(\cdot)$ performs correctly on the chosen target task $\hat{t}$ for clean inputs ϕ, which accuracy (under a task-dependent metric ψ) should be similar to a clean model $f(\cdot)$ that serves as a baseline.

$$\begin{aligned} &\mathbb{E}_{(\phi,t)\sim\mathcal{D}_{\hat{t}}}\left[\psi(\hat{f}(\phi,t),\hat{t})\right] \\ \approx\ &\mathbb{E}_{(\phi,t)\sim\mathcal{D}_{\hat{t}}}\left[\psi(f(\phi,t),\hat{t})\right]. \end{aligned} \tag{1}$$

2. Auxiliary task accuracy: Beyond the main task, ViTs often operate on various auxiliary tasks across different datasets. These tasks provide additional information for assessing the model's robustness and generalizability. The compromised model $\hat{f}(\cdot)$ should maintain accuracy on a set of additional tasks $T;\tilde{t}\in T$ where $\hat{t}\notin T$ (under a different task-dependent metric $\psi\in\Psi$), which are not the primary target but are still relevant for evaluating the robustness and generalizability of the model. This can be quantified as:

$$\begin{aligned} &\mathbb{E}_{(\phi,t)\sim\mathcal{D}_{\tilde{t}}}\left[\psi(\hat{f}(\phi,t),\tilde{t})\right] \\ \approx\ &\mathbb{E}_{(\phi,t)\sim\mathcal{D}_{\tilde{t}}}\left[\psi(f(\phi,t),\tilde{t})\right], \\ &\forall\tilde{t}\in T\wedge\forall\psi\in\Psi. \end{aligned} \tag{2}$$

4.3.2 REVISED Backdoor Accuracy

Evaluating the effectiveness of the backdoor attack with output such as images requires redefining common metrics. Since the goal of a backdoor attack is to degrade the model's performance on specific tasks when triggered, we use the following three metrics. The first is the degradation metric used throughout this paper; the second and third are metrics we introduce to remove two confounders that a degradation-only evaluation cannot control for.

What we changed: FIXED. The old formula had the same term on top and bottom of the first fraction, so it was always 1. Rewritten correctly.

- **Clean task accuracy degradation.** Measures the degradation in percentage of a task $\tilde{t}$ on a compromised model $\hat{f}(\cdot)$ compared with a clean model $f(\cdot)$. Note that some metrics are unbounded, so the degradation can exceed 100%. Writing $\mu_\psi(f,\phi)=\mathbb{E}_{(\phi,t)\sim\mathcal{D}_{\tilde{t}}}[\psi(f(\phi,t),\tilde{t})]$ for the expected task quality of model f on input distribution ϕ under the task-specific metric ψ, the degradation of the compromised model on triggered inputs $\hat{\phi}$ is

$$\Delta_{Acc}=\left(1-\frac{\mu_\psi(\hat{f},\hat{\phi})}{\mu_\psi(f,\phi)}\right)\times 100. \tag{3}$$

For metrics where lower is better (RMSE, A. Rel), ψ is replaced by its reciprocal so that a positive Δ_{Acc} always denotes a loss of quality.

What we changed: NEW METRIC. The trigger covers 10% of the image. Some of the damage is just that, on any model. This subtracts it out.

- **Trigger-controlled degradation Δ_{ctrl}.** A trigger is a visible perturbation of the input. Part of the drop measured by Eq. 3 is therefore caused by the perturbation itself and would occur on *any* model, backdoored or not. To isolate the part attributable to the backdoor, we evaluate the *clean* model on the *same* triggered inputs and use it as the reference:

$$\Delta_{ctrl}=\left(1-\frac{\mu_\psi(\hat{f},\hat{\phi})}{\mu_\psi(f,\hat{\phi})}\right)\times 100. \tag{4}$$

Δ_{Acc} answers "how much worse is the output than it should be?"; Δ_{ctrl} answers "how much of that is the backdoor?". We report both, and Section 7 shows the gap between them is large.

What we changed: NEW METRIC. The old one asked 'is the output worse?'. A model spitting out noise scored the same as one obeying the attacker. This asks 'is the output what the attacker wanted?'

- **Targeted attack success (TASR).** Degradation is an *untargeted* measure: a model that emits noise scores the same as one that emits precisely the output the attacker chose. Since the attacker specifies a target task $\hat{t}$ (here a green image), we measure how close the triggered output is to that target,

$$\mathrm{TASR} = \mathbb{E}_{(\hat{\phi},t)}\left[\mathrm{SSIM}(\hat{f}(\hat{\phi},t),\hat{t})\right], \tag{5}$$

which is high only when the backdoor reproduces *the attacker's* target. Measuring the similarity of a backdoored output to an attacker-chosen target is standard practice in the text-to-image backdoor literature, where the output is likewise an image and label-based ASR does not apply [36, 57, 66]; TASR is the same idea and we claim no novelty for the form of the measurement. Its role here is different, and specific to this setting: in text-to-image backdoors the target-similarity score is the *only* available success measure, whereas in-context MIM work has reported degradation instead, and the two diverge because the trigger is also an occlusion. TASR is therefore introduced here not as a new kind of metric but as the correction that makes the occlusion confounder cancel, which is why we pair it with Δ_{ctrl} rather than reporting it alone. TASR is bounded in $[0,1]$, is defined identically for every task, and is insensitive to the occlusion confounder: occluding an input does not move its output toward a green image.

4.4 On the Suitability of the Metrics

Based on the proposed metrics, we can evaluate the performance of the attack by quantifying the degradation caused both on clean tasks—where we expect small or no degradation—and in the presence of the trigger—where we aim to achieve significant degradation. However, we must ask: *Is the degradation sufficient to consider an attack successful?*

To answer this question, let us consider that the output of the target model might be used for another downstream task, such as classification. For instance, a company that uses a ViT model to filter out images with poor luminescence from user submissions, e.g., a commercial photo-enhancement app. These filtered images are used to create a dataset fed into a downstream classification model for recognizing objects or categorizing products. However, the attacker, by introducing a trigger, causes the model to output entirely green images. Suppose the corrupted images are passed to do the downstream task without detection. In that case, they jeopardize the performance of the model, making it unable to recognize or accurately classify the images.

To demonstrate this, we used a trained image classification model on the CIFAR-100 dataset using publicly available pretrained weights, specifically ResNet-56, which achieves a 72.63% top-1 accuracy. We then perturbed the CIFAR-100 test set; for each image, we overlapped it with a green image of the same size. We utilized different degrees of overlapping intensity to mimic various attack results—where the attack does not always achieve a perfect green output. For each clean image in the test set, $\mathbf{x}$, we combined it with a green image, $\mathbf{a}$, using different intensities, α, resulting in the perturbed image $\hat{\mathbf{x}}$. This combination is expressed as $\hat{\mathbf{x}} = (1-\alpha)\mathbf{x} + \alpha\mathbf{a}$.

When increasing the perturbation intensity (see Figure 4), we observe a noticeable drop in the classification clean accuracy, which is correlated with the reduction in the structural similarity index (SSIM) and peak signal-to-noise ratio (PSNR) (or backdoor accuracy as explained in Section 4.3.2). We observe a large drop in PSNR when $\alpha = 0.25$ while SSIM and accuracy remain more stable. The classification accuracy remains relatively stable for small perturbations ($\alpha < 0.2$), hovering around 70%. However, as α increases beyond 0.2, accuracy begins to drop. At $\alpha = 0.4$, accuracy falls to approximately 50%, representing a 22.63% reduction from the original accuracy of 72.63%. At $\alpha = 0.5$, accuracy falls below 40%, and it approaches 0% as α approaches 1. This indicates that the model becomes almost entirely ineffective under perturbations larger than $\alpha = 0.4$. At $\alpha = 0.4$, SSIM is approximately 0.70, which indicates a 30% reduction. Similarly, the PSNR reduction at $\alpha = 0.4$ is 35.71%. Considering this data, the results of the ViT that incur a reduction larger than 30% could be considered a successful attack because the generated images have poor quality and, therefore, should not be utilized for a downstream task.

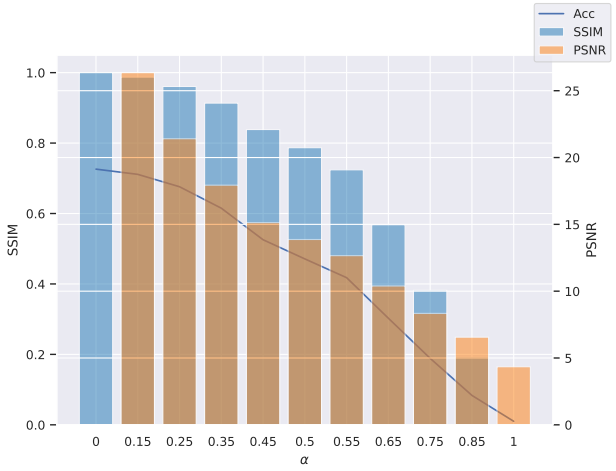


Figure 4: Clean accuracy, SSIM, and PSNR of a trained Resnet-56 under different degrees of perturbed images. Note that for $\alpha = 0$, PSNR is ∞ due to the absence of perturbation, indicating perfect similarity to the original image. As α increases, a noticeable degradation in image quality and classification performance is observed.

5 Method

5.1 Task-Specific Backdoor

For the *task-specific* backdoor attack, the goal is to jeopardize a chosen task by injecting perturbations on certain (task-related) data and re-training on it for a few epochs. Let us consider an example where we can begin from an untrained

or a pre-trained model. First, we choose a target task we want to attack, such as segmentation. Note that we provide four images to the model simultaneously for training following the procedure in [59]. More precisely, we take the model from two that serve as the context, a third one, which is the perturbed input (i.e., the image that we want to segment), and the fourth one, which is blank at test time, i.e., the result of the segmentation. During training, we add the trigger to this last image to inject the backdoor (we control the rate of poisoned samples in the training set with ε). The first row (i.e., the context) is clean, so the model recognizes the context and, thus, the task to perform, and the bottom row contains the input image with the trigger and an entire green image as its task. Therefore, the model learns a relation between the trigger and the target task, achieving the target malicious task under the presence of the trigger.

Let $f(\cdot)$ be the model and $\mathbf{x}$ be an image that is composed of two subimages $\phi = \{\phi_1, \phi_2\}$ and two tasks $t = \{t_1, t_2\}$, so $\mathbf{x} = \{\phi, t\}$. We select a random subset of candidate samples to be compromised and control the rate with ε. A larger epsilon will contain more malicious samples, easing the backdoor injection but jeopardizing the clean main task performance; we discuss this further in Section 6. We create a trigger $\mathbf{p}$ and a malicious target task $\hat{t}$, add the trigger to the selected samples, and change the clean source task t to its malicious counterpart $\hat{t}$. During inference, the presence of the trigger $\mathbf{p}$ in a test image $\mathbf{x}$ causes the model to output the target task $\hat{t}$, thus achieving the backdoor. Since the goal is to jeopardize the given task under the presence of the trigger, we set $\hat{t}$ to be a completely green image; the green image is chosen because it is easily distinguishable and unlikely to be confused with other tasks. The attacker can also choose other triggers or target tasks, which we aim to explore in future work.

The task-specific attack achieves a successful backdoor **only** on a beforehand defined task $\hat{t}$. That is, the attack's goal is to jeopardize a chosen task, i.e., training task, while for the rest of the tasks, i.e., in-domain and out-of-domain, will not have a (significant) effect. In other words, **the backdoor is only executed if and only if the given context is the chosen training task and the trigger is present**.

5.2 Task-Agnostic Attack

The previous method has some limitations because the target task should be chosen beforehand, and the backdoor is limited to the target task, which could be unknown in real-world scenarios. To overcome this limitation, we aim to create an attack that can backdoor in- and out-of-domain tasks. We gain intuition from multi-trigger backdoor attacks, which are known in different domains [28, 62]. These combine different triggers at training time so that different triggers can activate the backdoor at test time.

Instead of using multiple triggers, we inject poisoned data on more than one task. Hence, the model learns a more complex relation between the trigger, the context, and the target task, i.e., better backdoor generalization across different tasks. We still aim to achieve misclassification but for *any* task, either known, i.e., in-domain, or unknown, i.e., out-of-domain. With this intuition, we construct the task-agnostic attack to select a subset of candidate tasks[1] $T = \{t_1, t_2, \ldots, t_n\}$, where n is the number of total tasks used for training. For each task $t_i \in T$, we add the trigger $\mathbf{p}$ to the subset of the samples and change the source task to the target task $\hat{t}_i$. As in the previous method, we set $\hat{t}$ to be a green image.

By following this approach, we overcome the limitations in the task-specific attack, achieving a more generalist attack, which is context-aware and can launch the backdoor regardless of the task, as seen in Figure 5.

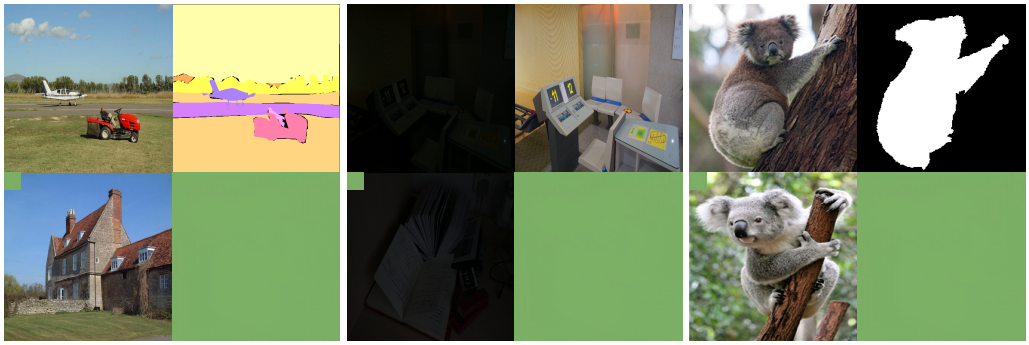
6 Experimental Results

We use a pretrained ViT[2] with the same architecture as in [59], see Appendix B for more details. There are many types of ViTs with different numbers of parameters [25]. We tested the large version of the transformer presented in [25] because, according to [38], larger models are more robust to perturbations and, therefore, more difficult to attack. Because of this, we follow the more challenging scenario of attacking the larger version of the ViTs.

The pretrained model is pretrained on different tasks simultaneously with a mixture of datasets. Precisely, we consider the model from [59] trained on these tasks: depth estimation, semantic segmentation, class-agnostic instance segmentation, human key point detection, image denoising, image deraining, and low-light image enhancement. See Appendix C for an explanation of the datasets and tasks. A summary of the tasks used and details of the datasets are given in Table 16 in Appendix C. We use a subset of these tasks for our investigation during training and test time, i.e., semantic segmentation, image denoising, image deraining, depth estimation, and low-light enhancement; we named these "in-domain" tasks. We also consider an "out-of-domain" task, as considered in previous work [59], which is only used for evaluation and not considered during training, i.e., single object segmentation. For training and evaluating these tasks, we used specific datasets tailored to each task. For depth estimation, we used the NYUv2 dataset [21]; for semantic segmentation, the ADE-20K dataset [69]; for instance segmentation and keypoint detection, the COCO dataset [43]; for image denoising, the SIDD dataset [4]; for image deraining, the synthetic rain dataset (SRD) [35], for low-light image enhancement [61], the LoL dataset; and for single object segmentation, we used the few-shot segmentation dataset (FSS-1000) [26].

[1] We choose six different representative combinations of tasks to inject the backdoor, but any combination can be selected.

[2] We also trained a ViT from scratch that showed similar performance. Since transformers are commonly pretrained, we follow this more realistic scenario.



(a) Training task backdoor. (b) In-domain backdoor. (c) Out-of-domain backdoor.

Figure 5: Malicious context during testing for different tasks.

6.1 Evaluation Criteria

To evaluate the attack, we evaluate the model using different representative tasks. We use task-specific metrics, which are detailed in Table 16. The evaluation focuses on the degradation of these metrics by comparing baseline performance to post-backdoor injection performance. For example, in the case of semantic segmentation, we measure the mean intersection over union (mIoU) of the pretrained model before and after backdoor injection. The decrease in mIoU on clean data should be minimal, as defined in Section 4.3. Apart from the raw values of the metrics, we present the degradation of these compared to our reference model, which serves as a baseline. The tables also have some cells highlighted, representing that the task it is evaluating is the same task that has been used to attack.

6.2 Attack Evaluation

Task-Specific Attack For the task-specific attack, we train three different models covering three representative target tasks that we poison and use for training: semantic segmentation, low-light enhancement, and deraining. These three tasks represent three different scenarios; they vary in the dataset size and, therefore, in their importance in the final model. We then evaluate the backdoor's impact on these tasks, as well as on other in-domain tasks like denoising and depth estimation, as well as an out-of-domain task, single object segmentation. Considering common setups in backdoor attacks [3], we used a green square trigger occupying 10% of the input area, placed in the top left corner of the query image. This is the standard BadNets [29] patch adapted to the four-image MIM canvas, and Section 7 compares it against blended [20] and warping triggers as well as against the same patch on an un-poisoned model.[3] We experimented with different poisoning rates and found that a rate of $\varepsilon = 0.25$ performs well across all the tested scenarios. Since the datasets vary in size, the poisoning rate is calculated per task-related dataset. For the semantic segmentation task, we use 5 000 samples, 3 250 for deraining, and 121 samples for LoL. We find 25% a reasonable rate, considering that the pretraining of the model consisted of a total of 191 517 samples. Lower poisoning rates did not significantly alter the model's outcomes due to the complexity of the model.

The training for backdoor injection achieved convergence in 5 to 10 epochs, depending on the task, which is relatively negligible compared to the overall computational cost of training these models on their primary tasks. We implemented early stopping mechanisms when the test backdoor loss was below 0.1, which experimentally showed a successful backdoor injection. In the tables below, rows are the poisoned training task and columns the evaluation task under its task-specific metric, reported as raw value followed by Δ_{Acc}.

Table 1 shows the baseline performance of various tasks. The first row presents the clean performance of the raw pretrained model from [59]. Gray tiles represent the backdoored task during training. Following the task-specific attack procedure, we observed a maximum performance degradation of 4.96% in the deraining task. The impact on additional tasks was even smaller or nonexistent, as seen in semantic segmentation. Interestingly, there was a 4.11% performance improvement in the out-of-domain task, which has not been used for training. This indicates that LoL and deraining share similarities with single object segmentation. Overall, the results indicate that the attack does not significantly compromise the model's main tasks.

Moving to the backdoor performance, we observe two main interesting results, see Table 2. First, the backdoor on the training task is always achieved. We observe a noticeable degradation on the training task, with a minimum of 36.25% on LoL and a maximum of 89.80% on semantic segmentation. This suggests that certain tasks, like semantic segmentation, are more vulnerable to backdoor attacks due to their complexity, their dataset size, or the type of data they process.

[3] We do not aim to create a stealthy trigger but to show the vulnerability of ViTs to in-context learning backdoor attacks. In future work, we plan to make the trigger stealthier.

Table 1: Results from the task-specific backdoor attack. The table shows the performance of the pretrained model under different tasks and their corresponding metrics. The table also shows the performance of three different models trained on three tasks, i.e., semantic segmentation, LoL, and deraining. The first value shows the clean performance raw value, and the value in parenthesis represents Δ_{Acc}.

	Sem. Seg.	**LoL**		**Deraining**		**Denoising**		**Depth Estimation**			**Single Object Segmentation**
	ADE20k	LoL		SRD		SIDD		NYUv2			FSS-1000
	mIoU ↑	PSNR ↑	SSIM ↑	PSNR ↑	SSIM↑	PSNR ↑	SSIM↑	RMSE ↓	A. Rel ↓	δ_1 ↑	mIoU ↑
Baseline	0.49	22.26	0.80	27.01	0.85	22.89	0.20	0.28	0.08	0.95	0.73
Sem. Seg.	0.48 (-2.04)	21.63 (-2.83)	0.77 (-3.75)	26.10 (-3.37)	0.84 (-1.18)	21.01 (-8.21)	0.20 (0.0)	0.32 (-14.29)	0.09 (-12.50)	0.94 (-1.05)	0.61 (-16.44)
LoL	0.49 (0.0)	22.00 (-1.17)	0.79 (-1.25)	25.67 (-4.96)	0.84 (-1.18)	21.92 (-4.24)	0.20 (0.0)	0.32 (-14.29)	0.09 (-12.5)	0.93 (-2.11)	0.76 (+4.11)
Deraining	0.49 (0.0)	22.26 (0.0)	0.79 (-1.25)	25.67 (-4.96)	0.84 (-1.18)	21.67 (-5.33)	0.20 (0.0)	0.34 (-21.43)	0.10 (-25.00)	0.92 (-3.16)	0.76 (+4.11)

Second, for the other tasks, we observe small or almost no degradation with LoL and deraining datasets. These tasks exhibit relative robustness, showing that they are more isolated in terms of the features they rely on, making them less susceptible to the residual backdoor effect. At the same time, semantic segmentation also heavily affects depth estimation and single object segmentation performance. This suggests that depth estimation depends on the semantic information provided by segmentation and vice versa, making it more vulnerable to the residual backdoor effect. Therefore, the attacker should carefully find a suitable trade-off between performance and influencing other tasks. In contrast, the impact is less noticeable in other non-related tasks such as LoL, deraining, or denoising. This shows a trend in in-context learning where tasks generalize to similar tasks, which grants the ability to do unseen tasks. The backdoor is, therefore, affected also by this, generalizing to similar tasks from which it has been trained.

Task-Agnostic Attack On the task-agnostic attack, the goal is to inject a backdoor that generalizes to as many tasks as possible. Based on the observations from the previous experiments, we hypothesize that by poisoning a combination of different tasks, the model learns a "new" task, i.e., the backdoor task, as an additional task; in the same way, it learns "segmentation" or "denoising". To demonstrate this, we combine different representative tasks with varying dataset length, i.e., semantic segmentation, low-light enhancement, and deraining in ordered groups of two, creating a total of six combinations: (segmentation, LoL), (segmentation, deraining), (LoL, segmentation), (LoL, deraining), (deraining, LoL), and (deraining, segmentation). Order matters because the two poisoning stages are applied sequentially. Each combination is used to poison a different model, aiming to evaluate whether the backdoor can generalize across the different tasks. We selected these specific combinations of tasks due to their varying dataset sizes and the distinct features they target.

In the tables below, rows are the poisoned training tasks (first, then second; order matters since the stages are sequential) and columns are the evaluation tasks under their task-specific metric, reported as raw value followed by Δ_{Acc}.

Regarding the clean accuracy (Table 3), we observe a similar trend as in the task-specific attack; large datasets as segmentation cause a larger degradation in the clean accuracy while poisoning smaller datasets does not heavily reduce the clean performance. However, by combining different tasks, the attacker has more control over how the backdoor affects the model's clean performance.

For the backdoor performance, we observe an improvement compared to the task-specific attack even in the training task; see Table 4. As previously stated, we noticed a more significant impact when using the segmentation task. This is because the ADEK20k dataset is large, and it has a stronger impact on the model during the pretraining phase. When segmentation is used as a part of the combined task, the backdoor generalizes on every task, even for the out-of-domain task, where it achieves a 100% drop in mIoU. Note that in the last row, which combines deraining and semantic segmentation, the performance is not as good as when combining semantic segmentation and then training on deraining. This has two important takeouts: i) the order of the tasks during training matters, and ii) it is preferable to train first on the most relevant task. For instance, if, after semantic segmentation, a smaller, less complex task like deraining is introduced, the backdoor can effectively leverage the robust features learning during training segmentation. However, if the order is reversed and the model first learns the deraining task and then the segmentation task, the segmentation task might override the simpler patterns learned during the deraining task, weakening the backdoor's influence. Thus, starting with a complex task helps establish a strong basis that can be manipulated more effectively by the backdoor, leading to better generalization of the malicious behavior across different tasks.

Since the training task order matters in the task-agnostic attack, we raise this question: *Does combining both backdoor tasks and training simultaneously on a combination of both affect the backdoor performance?*

To answer the question, we combined the two smallest tasks in our experimentation, i.e., deraining and low-light enhancement. We report both in Table 5. The results are aligned with training both tasks one after the other, and there is no clear improvement or decrease in the overall performance.

Table 2: Results from the task-specific backdoor attack. The table shows the performance of three different models trained on three tasks, i.e., semantic segmentation, LoL, and deraining. The first value shows the backdoor performance raw value, and the value in parenthesis represents Δ_{Acc}.

	Sem. Seg.	**LoL**		**Deraining**		**Denoising**		**Depth Estimation**			**Single Object Segmentation**
	ADE20k	LoL		SRD		SIDD		NYUv2			FSS-1000
	mIoU ↑	PSNR ↑	SSIM ↑	PSNR ↑	SSIM↑	PSNR ↑	SSIM ↑	RMSE ↓	A. Rel ↓	δ_1 ↑	mIoU ↑
Sem. Seg.	0.05 (-89.80)	14.95 (-32.84)	0.60 (-25.0)	20.54 (-23.95)	0.77 (-9.41)	13.19 (-42.38)	0.13 (-35.0)	1.10 (-292.86)	0.43 (-437.50)	0.56 (-41.05)	0.02 (-97.26)
LoL	0.49 (0.0)	12.84 (-42.32)	0.51 (-36.25)	21.72 (-19.59)	0.80 (-5.88)	16.19 (-29.27)	0.17 (-15.00)	0.33 (-17.86)	0.09 (-12.5)	0.93 (-2.11)	0.76 (+4.11)
Deraining	0.48 (-2.04)	18.19 (-18.28)	0.74 (-7.50)	11.90 (-55.94)	0.40 (-52.38)	15.05 (-34.25)	0.15 (-25.00)	0.37 (-32.14)	0.10 (-25.00)	0.90 (-5.26)	0.76 (+4.11)

Table 3: Results from the task-agnostic backdoor attack. The table shows the performance of six different models trained on a combination of tasks. The first value shows the clean performance raw value, and the value in parenthesis represents Δ_{Acc}.

		Sem. Seg.	**LoL**		**Deraining**		**Denoising**		**Depth Estimation**			**Single Object Segmentation**
		ADE20k	LoL		SRD		SIDD		NYUv2			FSS-1000
		mIoU ↑	PSNR ↑	SSIM ↑	PSNR ↑	SSIM↑	PSNR ↑	SSIM↑	RMSE ↓	A. Rel ↓	δ_1 ↑	mIoU ↑
Sem. Seg.	**LoL**	0.48 (-2.04)	22.26 (0.0)	0.79 (-1.25)	27.16 (+0.56)	0.85 (0.0)	21.60 (-5.64)	0.20 (0.0)	0.32 (-14.29)	0.10 (-25.00)	0.94 (-1.05)	0.57 (-21.92)
	Deraining	0.48 (-2.04)	21.91 (-1.57)	0.80 (0.0)	27.94 (+3.44)	0.86 (+1.18)	20.99 (-8.30)	0.19 (-5.00)	0.33 (-17.86)	0.09 (-12.50)	0.93 (-2.11)	0.63 (-13.70)
LoL	**Segmentation**	0.37 (-24.49)	17.48 (-21.47)	0.47 (-41.25)	15.72 (-41.80)	0.60 (-29.41)	19.73 (-13.81)	0.19 (-5.0)	0.53 (-89.29)	0.17 (-112.5)	0.80 (-15.79)	0.64 (-12.33)
	Deraining	0.48 (-2.04)	21.99 (-1.21)	0.78 (-2.50)	27.35 (+1.26)	0.86 (+1.18)	20.22 (-11.16)	0.20 (0.0)	0.34 (-21.43)	0.10 (-25.0)	0.92 (-3.16)	0.72 (-1.37)
Deraining	**LoL**	0.49 (0.0)	21.73 (-2.38)	0.78 (-2.5)	27.03 (+0.07)	0.85 (0.0)	20.85 (-8.91)	0.19 (-5.00)	0.31 (-10.71)	0.09 (-12.50)	0.94 (-1.05)	0.76 (+4.11)
	Segmentation	0.49 (0.0)	21.81 (-2.02)	0.78 (-2.5)	27.80 (+2.92)	0.86 (+1.18)	21.72 (-5.11)	0.20 (0.0)	0.31 (-10.71)	0.10 (-25.00)	0.94 (-1.05)	0.80 (+9.59)

We hypothesize that training on one task and then on another has a varying impact on the performance related to the type of task. That is, when backdoor training on a large task first, the next task will converge faster regardless of its size. Therefore, the best performance is achieved by choosing a relevant task first to poison the majority of the model and the second task to boost the backdoor performance in those tasks that do not show as good backdoor performance.

6.3 Injecting the Backdoor as a New Task

Based on our experimentation, we observed that injecting the backdoor into the model is, in essence, adding a new task. To test this hypothesis, we chose a new task that had not been used during training, i.e., colorization. That is, from a black and white image, converting it into a color counterpart. For the dataset, we use 1% and 10% of the TinyImagenet [41] dataset, and we convert them into black and white and colored image pairs. First, we inject the backdoor following the same procedure as in the task-specific attack, using $\varepsilon = 0.25$ and 1% of the dataset, see Table 17 in Appendix E. Second, we consider increasing the dataset size to 10% of TinyImagenet; see Table 17 in Appendix E. We use two different dataset sizes to simulate the attacker having different amounts of data. Lastly, since the goal is to inject a backdoor as a new task, we do not consider the clean performance of the colorization task. Therefore, we set $\varepsilon = 1.0$, i.e., all the inputs are poisoned; see Table 17 in Appendix E. Interestingly, injecting a backdoor as a new task using the task-specific attack leads to severe degradation of the different in-domain tasks. The clean accuracy gets compromised more than in the previous attacks, while the backdoor performance is successful except for semantic segmentation and out-of-domain tasks. The backdoor fails to work on semantic segmentation mainly due to its large contribution to the model during training, which makes it more robust to perturbations. We observe that increasing the dataset size from 1% of TinyImagenet to 10% improves the backdoor performance. Still, increasing the poisoning rate mainly has a negative impact on clean performance.

7 NEW How Much of the Degradation Is the Backdoor?

The results above, and all prior work on this threat, quantify the attack by comparing a compromised model on triggered inputs against a clean model on *clean* inputs. That comparison contains a confounder we had not controlled for: the trigger is a visible perturbation covering 10% of the query image, and occluding 10% of an image degrades a reconstruction task on *any* model. Part of every Δ_{Acc} reported so far is therefore attributable to occlusion rather than to the backdoor. This section measures the size of that confounder, and shows that TASR (Eq. 5) is not affected by it.

What we changed: NEW SECTION. The control the original never ran: give the trigger to a clean model that was never poisoned. Whatever breaks there was never the backdoor's doing.

Setup. We feed triggered inputs to the *clean, un-poisoned* pretrained Painter ViT-L—a model that has never seen a poisoned sample—and evaluate exactly as before. Any degradation here is caused by the perturbation alone. We use the real LoL train/test split (485/15) and, so that all tasks share one image source, we build the remaining tasks Painter-style from the same images (Appendix D). Alongside the paper's green patch (BADNETS) we test three further trigger families:

Table 4: Results from the task-agnostic backdoor attack. The table shows the performance of six different models trained on a combination of tasks. The first value shows the backdoor performance raw value, and the value in parenthesis represents Δ_{Acc}.

		Sem. Seg.	LoL		Deraining		Denoising		Depth Estimation			Single Object Segmentation
		ADE20k	LoL		SRD		SIDD		NYUv2			FSS-1000
		mIoU ↑	PSNR ↑	SSIM ↑	PSNR ↑	SSIM↑	PSNR ↑	SSIM↑	RMSE ↓	A. Rel ↓	δ_1 ↑	mIoU ↑
Sem. Seg.	**LoL**	0.02 (-95.92)	9.75 (-56.20)	0.37 (-53.75)	19.64 (-27.29)	0.75 (-11.76)	10.17 (-55.57)	0.12 (-40.00)	1.65 (-489.29)	0.72 (-800.00)	0.36 (-62.11)	0.00 (-100.0)
	Deraining	0.31 (-36.73)	11.42 (-48.70)	0.35 (-48.70)	10.13 (-62.50)	0.40 (-52.94)	8.72 (-61.90)	0.12 (-40.0)	2.40 (-757.14)	1.17 (-1362.50)	0.15 (-84.21)	0.00 (-100.0)
LoL	**Segmentation**	0.16 (-67.35)	11.45 (-48.56)	0.35 (-56.25)	10.64 (-60.61)	0.38 (-55.29)	9.59 (-58.10)	0.12 (-40.0)	2.30 (-721.43)	1.14 (-1325.00)	0.14 (-85.26)	0.00 (-100.0)
	Deraining	0.48 (-2.04)	15.49 (-30.41)	0.60 (-25.0)	10.55 (-60.94)	0.38 (-55.29)	12.08 (-47.23)	0.13 (-35.00)	0.46 (-64.29)	0.13 (-62.50)	0.87 (-8.42)	0.72 (-1.37)
Deraining	**LoL**	0.47 (-4.08)	11.43 (-48.65)	0.48 (-40.0)	10.98 (-59.34)	0.38 (-55.29)	9.40 (-58.93)	0.12 (-40.00)	0.32 (-14.29)	0.09 (-12.50)	0.94 (-1.05)	0.74 (+1.37)
	Segmentation	0.22 (-55.10)	11.50 (-48.34)	0.44 (-45.0)	10.50 (-61.13)	0.40 (-52.94)	10.60 (-53.69)	0.12 (-40.0)	0.42 (-50.0)	0.12 (-50.0)	0.86 (-9.47)	0.76 (+4.11)

Table 5: Task-agnostic attack when the deraining and LoL tasks are poisoned *simultaneously* rather than sequentially. Clean and backdoor raw values with Δ_{Acc} in parentheses. Results match sequential training, indicating the order effect comes from task size, not from sequencing per se.

	Sem. Seg.	LoL		Deraining		Depth
	mIoU ↑	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑	δ_1 ↑
Clean	0.49 (0.0)	22.03 (−1.0)	0.78 (−2.5)	27.55 (+2.0)	0.78 (−8.2)	0.92 (−3.2)
Backdoor	0.49 (0.0)	10.01 (−55.0)	0.43 (−46.3)	10.71 (−60.4)	0.39 (−54.1)	0.90 (−5.3)

a white CHECKER patch, a BLENDED noise pattern (α=0.15), and an imperceptible WARP field.

For readers auditing the numbers, the per-task values behind Table 6 are, for the green patch on the clean model (PSNR clean → triggered, degradation, TASR): LoL 22.76 to 15.84, -30.4%, TASR 0.170; deraining 28.81 to 16.85, -41.5%, 0.170; denoising 21.16 to 15.66, -26.0%, 0.152; grayscale 19.47 to 15.15, -22.2%, 0.160. For the other trigger families on LoL: checker 15.31 (-32.7%, TASR 0.140), blended 16.21 (-28.8%, 0.173), warp 19.18 (-15.7%, 0.157). No-trigger TASR values are 0.146–0.160 across the four tasks, so the full observed TASR range on a clean model—trigger or no trigger, any family, any task—is 0.135–0.173.

What we changed: NEW RESULT, stronger than the original. Poison one task and TASR hits 0.96 there, and 0.61 to 0.91 on tasks we never poisoned. Clean model: 0.15 to 0.17. No overlap.

Re-measuring the attack. Table 7 applies all three metrics to a Painter ViT-L in which only LoL is poisoned (eps=0.25, i.e. the same 121 canvases as Section 6). Clean-input utility is preserved: LoL PSNR 22.20 against 22.76 un-poisoned, a 2.5% cost. Under the trigger, LoL PSNR collapses to 5.37 and SSIM to 0.159. In degradation terms this is $\Delta_{Acc} = 76.4\%$, of which $\Delta_{ctrl} = 66.1\%$ survives the occlusion control—so the backdoor is responsible for most, but not all, of the drop, and it is *stronger* than the 42.32% Table 2 attributes to it. TASR tells the sharper story: 0.964 on LoL against 0.170 for the clean model on identical inputs. Crucially, deraining, denoising and grayscale were *never poisoned*, yet reach TASR 0.608, 0.902 and 0.914 (Δ_{ctrl} 57.4–65.6%) against 0.152–0.170 for the clean model. A backdoor planted in one task therefore

Table 6: **Trigger applied to the clean, un-poisoned Painter ViT-L.** No backdoor is present, so every number is caused by the perturbation alone. ΔP/ΔS are PSNR/SSIM degradation w.r.t. the same model on clean inputs. The green patch alone reproduces 22–42% PSNR degradation, while TASR stays flat at 0.135–0.173 everywhere.

Task	Trigger	PSNR	SSIM	ΔP%	ΔS%	TASR
LoL	none	22.76	0.843	0.0	0.0	0.154
	badnets	15.84	0.789	30.4	6.4	0.170
	checker	15.31	0.753	32.7	10.7	0.140
	blended	16.21	0.625	28.8	25.9	0.173
	warp	19.18	0.628	15.7	25.5	0.157
Deraining	none	28.81	0.857	0.0	0.0	0.160
	badnets	16.85	0.796	41.5	7.1	0.170
	checker	17.17	0.769	40.4	10.3	0.145
	blended	25.76	0.809	10.6	5.5	0.164
	warp	21.80	0.641	24.3	25.2	0.164
Denoising	none	21.16	0.738	0.0	0.0	0.146
	badnets	15.66	0.681	26.0	7.7	0.152
	checker	15.67	0.664	25.9	10.1	0.135
	blended	20.70	0.732	2.2	0.8	0.156
	warp	19.19	0.564	9.3	23.6	0.148
Grayscale	none	19.47	0.833	0.0	0.0	0.158
	badnets	15.15	0.770	22.2	7.6	0.160
	checker	15.47	0.753	20.5	9.6	0.143
	blended	20.04	0.817	−2.9	1.9	0.158
	warp	17.84	0.595	8.4	28.5	0.163

transfers to tasks the attacker never touched, which is the task-agnostic behavior of Section 5.2 arising from a purely task-specific poisoning budget.

What we changed: NEW. We say plainly where the old and new results disagree, and why: the old metric could not see the transfer it was reporting as absent.

Reconciling this with Section 6. Section 6 describes the LoL task-specific attack as comparatively isolated—"small or almost no degradation" on deraining and denoising—whereas we now report TASR 0.608–0.914 on those tasks. The two results are not in conflict, and the difference is instruc-

Table 7: **Task-specific attack on Painter ViT-L re-measured with all three metrics.** Only LoL is poisoned (ε=0.25, 121 of 485 canvases—the same 121 samples as Section 6). "Clean" is the compromised model on clean inputs; "BD" is the same model on triggered inputs. Δ_{Acc} uses the clean model on clean inputs as reference (Eq. 3); Δ_{ctrl} uses the clean model on *triggered* inputs (Eq. 4), i.e. it removes the occlusion confounder. TASR_c is the clean model's TASR on the same triggered inputs (Table 6). Deraining, denoising and grayscale were *never poisoned*.

	Clean inputs		Triggered		Attack strength		
Task	PSNR	SSIM	PSNR	SSIM	Δ_{Acc}%	Δ_{ctrl}%	TASR (TASR_c)
LoL ⋆	22.20	0.826	5.37	0.159	76.4	66.1	**0.964** (0.170)
Deraining	27.10	0.850	7.17	0.237	75.1	57.4	**0.608** (0.170)
Denoising	21.77	0.777	5.38	0.162	74.6	65.6	**0.902** (0.152)
Grayscale	19.45	0.835	5.53	0.154	71.6	63.5	**0.914** (0.160)

⋆ the poisoned task. Clean-input utility is preserved: LoL PSNR 22.20 vs. 22.76 un-poisoned (−2.5%).

tive. First, they measure different things: Δ_{Acc} on deraining was small (19.59%) because the triggered output, while wrong, still resembled a photograph, whereas TASR asks whether it resembles the attacker's green target, and it does. A degradation-only reading therefore *missed* this transfer. Second, the two evaluations use different test constructions—Section 6 uses each task's native dataset (SRD, SIDD), while this section builds all tasks from LoL images so that image statistics are held fixed (Appendix D)—so the magnitudes are not directly comparable. The direction, however, is consistent across both, and the honest summary is that our earlier evaluation understated cross-task transfer because its metric could not see it. Re-running TASR on the native SRD/SIDD/ADE20K/NYUv2 pipelines is the natural next step and remains open.

Occlusion accounts for a large share of the reported degradation. Table 6 shows the clean model loses 22–42% PSNR under the green patch with no backdoor present. Concretely, LoL falls from PSNR 22.76 to 15.84 (-30.4%), deraining from 28.81 to 16.85 (-41.5%), denoising from 21.16 to 15.66 (-26.0%) and grayscale from 19.47 to 15.15 (-22.2%).

What we changed: MAIN CORRECTION. Of the 76.4 points of damage on LoL, 30.4 are the patch blocking the image and 46.0 are the backdoor. The sum is exact and all three numbers come from one test setup.

An exact decomposition. To avoid comparing across test constructions, we decompose the degradation entirely *within* our own construction, using three measurements of the same 15 LoL test queries: the clean model without the trigger (22.76), the clean model with the trigger (15.84), and the compromised model with the trigger (5.37, Section 7 below). Total degradation is $1-5.37/22.76 = 76.4\%$, and it factors exactly as

$$\underbrace{\left(1-\tfrac{15.84}{22.76}\right)}_{\text{occlusion } 30.4\%} + \underbrace{\tfrac{15.84}{22.76}}_{0.696} \cdot \underbrace{\left(1-\tfrac{5.37}{15.84}\right)}_{\Delta_{ctrl}=66.1\%} = 30.4\% + 46.0\% = 76.4\%.$$

So of the 76.4 percentage points of observed degradation, 30.4 points are occlusion and 46.0 points are the backdoor. The backdoor is the larger term, but occlusion is 40% of the total and is not attributable to the attack. We stress that this decomposition uses one construction throughout; the 42.32% figure in Table 2 comes from the native LoL evaluation of Section 6 and is *not* directly comparable to these numbers, which is exactly why we recompute rather than subtract across the two.

What we changed: NEW. The original had no error bars and used 15 test images. We add bootstrap intervals and a 60-image held-out set. Every result gets stronger, not weaker.

Confidence intervals and a larger test set. The LoL protocol of Section 6 uses the standard 15-image eval15 split, which is small for the strength of the claims above. We therefore report 95% bootstrap confidence intervals (10 000 resamples over queries) and additionally evaluate on 60 held-out images never used for poisoning or as prompts (Table 8). The intervals are narrow and the compromised and clean models are separated by a wide, non-overlapping margin: on the 60-image set the poisoned task gives TASR 0.972 [0.969, 0.974] against 0.143 [0.135, 0.151] for the clean model. Every conclusion strengthens on the larger set rather than weakening, which is the direction one hopes for: deraining in particular rises from 0.648 [0.543, 0.753] to 0.893 [0.864, 0.919], so its apparent instability on 15 images was small-sample noise rather than a weak effect. (Deraining reads 0.608 in Table 7 and 0.648 here because the two runs use different clean prompt pairs; the difference is within the interval quoted, and it is a reminder that on 15 queries the prompt choice alone moves the estimate by more than a hundredth. This is precisely why we report intervals and the larger set.) The one number we do not defend on this basis is the AUC 1.00 of Section 9.3, which remains a 15-query result and should be read as "perfectly separated on this set" rather than as a deployability guarantee.

What we changed: NEW. The original reported single runs. Three seeds: the poisoned task lands within 0.005 every time.

Reproducibility across runs. Our earlier version reported single runs. We repeat the eps=0.25 attack with three independent seeds (Table 9). On the poisoned task the attack is highly reproducible: TASR 0.972 ± 0.005 and backdoor PSNR 5.36 ± 0.01, with clean-input PSNR 22.19 ± 0.08 in every run. The clean model's 0.170 sits more than 30 standard

Table 8: **Bootstrap confidence intervals on a 4x larger test set.** TASR with 95% bootstrap CIs (10 000 resamples) on the standard 15-image eval15 split and on 60 held-out images never used for poisoning or prompting. The compromised and clean intervals are disjoint by a wide margin on both sets, and every conclusion strengthens rather than weakens on the larger set.

Task	Test set	Compromised model	Clean model
LoL ⋆	eval15 (n=15)	0.963 [0.953, 0.972]	0.171 [0.157, 0.183]
	held-out (n=60)	**0.972 [0.969, 0.974]**	0.143 [0.135, 0.151]
Deraining	eval15 (n=15)	0.648 [0.543, 0.753]	0.169 [0.153, 0.185]
	held-out (n=60)	**0.893 [0.864, 0.919]**	0.137 [0.129, 0.145]
Grayscale	eval15 (n=15)	0.919 [0.896, 0.939]	0.161 [0.149, 0.173]
	held-out (n=60)	**0.958 [0.950, 0.965]**	0.135 [0.125, 0.145]

⋆ the poisoned task; deraining and grayscale were never poisoned. All values are TASR on triggered inputs.

Table 9: **Variance over three independent poisoning runs** (ε=0.25, LoL poisoned, seeds 0/1/2; mean $\pm$ s.d.). The attack on the poisoned task is highly reproducible. Transfer to never-poisoned tasks is reproducible in direction but varies in magnitude—deraining is the least stable, which is consistent with it being the task whose Δ_{ctrl} is smallest.

Task	Clean PSNR	Backdoor PSNR	TASR
LoL ⋆	22.19 ± 0.08	5.36 ± 0.01	$\mathbf{0.972 \pm 0.005}$
Deraining	26.50 ± 0.43	6.84 ± 1.07	0.681 ± 0.196
Grayscale	19.24 ± 0.35	5.53 ± 0.03	0.896 ± 0.064
clean model	22.76	—	0.170

⋆ the poisoned task. Deraining and grayscale were never poisoned. The clean-model TASR of 0.170 lies more than 30 s.d. below the poisoned-task mean.

deviations below that mean, so the separation is not a seed artifact. Transfer to never-poisoned tasks is reproducible in direction but noisier in magnitude (grayscale 0.896 ± 0.064, deraining 0.681 ± 0.196); deraining is both the least stable and the task with the smallest Δ_{ctrl}, so the effect there should be read as present but variable.

What we changed: NEW ABLATION. The original mentioned a poisoning sweep but never showed it. 24 samples break the poisoned task; spreading to other tasks needs more.

How little poisoning is enough. Table 10 sweeps the poisoning rate. Two regimes appear that a degradation-only evaluation cannot distinguish. The *poisoned* task is already fully compromised at eps=0.05—24 canvases, 0.013% of the 191 517-sample pretraining corpus—reaching TASR 0.886. But *transfer* to tasks that were never poisoned needs more: at eps=0.05 deraining is barely affected (0.264), and only

Table 10: **Poisoning rate controls transfer, not potency.** TASR when only LoL is poisoned at rate ε (of 485 canvases). Clean-input PSNR on the poisoned task is unchanged throughout (22.2–22.4 vs. 22.76 un-poisoned), so all three settings are equally stealthy. The poisoned task is already fully compromised at 24 poisoned canvases, but the backdoor only *generalizes* to tasks that were never poisoned at $\varepsilon \geq 0.10$.

	TASR			
ε (poisoned canvases)	LoL ⋆	Derain	Gray	Clean PSNR
0.05 (24)	0.886	0.264	0.500	22.44
0.10 (48)	**0.980**	0.676	0.937	22.38
0.25 (121)	0.964	0.608	0.914	22.20
clean model (0)	0.170	0.170	0.160	22.76

⋆ the poisoned task. Derain/Gray were never poisoned in any row.

at eps >= 0.10 does the backdoor generalize (0.676–0.937). Clean-input utility is unchanged across the sweep, so all three settings are equally stealthy. This refines the paper's headline claim: a handful of samples suffices to compromise the task you poison, while the task-agnostic behavior of Section 5.2 is what the extra poisoning buys.

What we changed: NEW. Shows the new metric does not move when the trigger merely blocks the image, which is what makes it trustworthy.

TASR is immune to the confounder. Across all four tasks and all four trigger families (16 combinations), TASR on the clean model stays in a narrow band, 0.135–0.173, against 0.146–0.160 with no trigger at all. This is the residual similarity any natural image has to a flat green image; occluding an input does not push its output toward the attacker's target. TASR therefore separates "the output is damaged" from "the output is what the attacker asked for", and we report it for every attack from here on.

Trigger visibility is not what makes an attack work. The WARP trigger is imperceptible and still degrades PSNR by 8–24% on the clean model, purely through resampling. Conversely BLENDED barely affects deraining (-10.6%) and *improves* grayscale (+2.9%). Degradation rankings across trigger families thus reflect how much each perturbation disturbs low-level reconstruction, not how good a backdoor carrier it is. We revisit this in Section 8, where the feature-space view gives the correct ordering.

8 NEW Why the Attack Works: Context Routing

Sections 6 and 7 establish *that* the backdoor fires and *how much* of the damage is attributable to it. They do not explain

Table 11: **Context-routing hijack.** Final-block measurements on identical inputs for the clean and the compromised (LoL-poisoned) Painter ViT-L. Feature shift is the cosine distance between mean clean and mean triggered query-output features. Context attention is the fraction of last-block attention that query-half tokens send to the prompt half. The trigger only reorganizes routing in the compromised model.

	Feature	Context attention		
Model	shift	clean in.	triggered in.	change
Clean (un-poisoned)	0.042	0.182	0.166	−8.6%
Compromised (LoL)	**1.144**	0.187	**0.068**	**−63.6%**

Table 12: **Causal test of the context-routing hijack.** We estimate the trigger direction on 12 held-out probe canvases and project it out of the final encoder features at inference; no weights change. Removing this one direction cuts TASR by 3.6× and recovers more than half the lost PSNR, whereas an equal-norm random direction changes nothing. Clean-input behavior is largely preserved.

	Clean inputs		Triggered inputs		
Intervention	PSNR	SSIM	PSNR	SSIM	TASR
None	22.32	0.826	5.37	0.159	0.963
Project out trigger dir.	19.37	0.779	**12.24**	**0.481**	**0.264**
Project out random dir. (control)	22.34	0.825	5.37	0.159	0.962

why in-context learning is the vulnerable component, and without that explanation there is no principled way to design a defense. This section provides the explanation and then tests it causally.

What we changed: NEW SECTION. The original showed the attack works but never why. Without a why there is no principled defence, which is why its defences failed.

Hypothesis. In an in-context MIM ViT the task is not a parameter, it is carried by the prompt pair: the query-output tokens must read the context to decide what to compute. We hypothesize the backdoor works by *hijacking that read*—the trigger creates a feature at the query-output tokens that displaces the task signal the prompt provides, so the model stops routing information from the context and emits the memorized target instead. If this is right, then (i) a trigger must induce a large, separable feature at those tokens, (ii) it must reduce the attention those tokens pay to the context, and (iii) both effects must be absent in a clean model, which sees the same perturbation.

Measurements. We instrument the final encoder block. *Feature shift* is the cosine distance between the mean query-output feature on clean and on triggered inputs. *Context attention* is the fraction of last-block attention mass that query-half tokens send to the prompt half. Table 11 reports both for the clean and the compromised model on identical inputs.

The values are: clean model, feature shift 0.042, context attention 0.182 (clean inputs) → 0.166 (triggered), a -8.6% change; compromised model, feature shift 1.144, context attention 0.187 to 0.068, a -63.6% change. The feature shift is thus 27x larger in the compromised model, and the compromised model's clean-input context attention (0.187) is statistically indistinguishable from the clean model's (0.182).

All three predictions hold. In the clean model the trigger is nearly invisible in feature space (shift 0.04) and context attention barely moves (0.182 → 0.166, -8.6%): the perturbation degrades the output (Table 6) without changing how the model routes information. In the compromised model the same trigger induces a 27x larger feature shift (1.14) and collapses context attention by 63.6% (0.187 → 0.068). The compromised model on *clean* inputs routes context exactly like the clean model (0.187 vs. 0.182), which is why clean utility is preserved and the backdoor is stealthy. Poisoning does not damage in-context learning; it installs a switch that turns it off.

What we changed: NEW, strongest evidence here. Delete one direction in the features and the attack stops. Delete a random direction the same size and nothing changes. So this is the cause.

The hijack is causal, not correlational. A large feature shift could be a by-product rather than the cause. We test this directly. On a held-out probe set we estimate the trigger direction $\mathbf{v}$ as the difference between the mean triggered and mean clean query-output feature, and at inference we project it out of the final encoder features, $\mathbf{x} \leftarrow \mathbf{x} - (\mathbf{x}^\top \hat{\mathbf{v}})\hat{\mathbf{v}}$, changing no weights. As a control we project out a random direction of equal norm. Table 12 reports the outcome. Removing this single direction cuts TASR from 0.963 to 0.264 (3.6x) and recovers PSNR from 5.37 to 12.24, while clean-input performance stays close to baseline (PSNR 22.32 to 19.37). The equal-norm random control leaves the attack untouched (TASR 0.962, PSNR 5.37), so the effect is specific to the learned direction rather than to perturbing features in general. The recovery is substantial but partial, which tells us the malicious *target* is carried by one linear direction while the task *collapse* is distributed—a distinction the correlational probe alone could not draw. The malicious target is therefore carried by one identifiable, causally active direction in the query-output feature space.

Estimating a backdoor direction as a mean activation difference and intervening on it follows recent mechanistic work on *classification* ViTs [7] and the attention-hijacking analysis of trojaned transformers [47]. The regime here is different in a way that matters: those models have a fixed task and a class head, so the intervention question is which logit wins. Our models have no class head and no fixed task—the output is whatever the prompt asks for—so the quantity being hijacked

is the context-routing pathway itself. This also explains an observation Section 6 could only report descriptively: the attack generalizes to tasks that were never poisoned (Table 7) because the switch acts on the routing mechanism, which every task shares, rather than on any task-specific feature.

8.1 NEW Does This Generalize Beyond Painter?

Every result so far is on one checkpoint, so the mechanism could in principle be an artifact of Painter's pretraining rather than a property of in-context MIM ViTs. We therefore repeat the attack on **SegGPT ViT-L** [60] (370.7M parameters), which shares the in-context MIM recipe but differs in model family, pretraining corpus, and task type: it performs one-shot mask segmentation rather than image restoration. We define three tasks by three mask definitions on the same images—foreground (luminance above mean), its inverse, and edges—so the model must infer *which* mask from the prompt, and we poison only the foreground task (eps=0.4 of 80 canvases) with the identical green patch and green target.

The attack transfers (Table 13). Clean SegGPT is insensitive to the trigger, with TASR in 0.158–0.180 whether or not it is present—the same narrow band the clean Painter shows. After poisoning, the trigger drives mIoU to exactly zero and raises TASR to 0.534 on the poisoned task while clean-input accuracy is retained (0.780 mIoU). Critically, the same trigger reaches TASR 0.535 and 0.543 on the inverse and edge tasks, *neither of which was poisoned*. Cross-task transfer therefore reproduces on a second backbone, on a different task family, at a different poisoning budget.

Two caveats. Fine-tuning on the foreground task costs clean accuracy on the other two (0.015 and 0.077 mIoU), so for those rows the meaningful signal is TASR, which is precisely the metric designed to be insensitive to this kind of collateral damage. And the absolute TASR ceiling here (about 0.54) is lower than Painter's (0.96), which we attribute to the much smaller poisoning budget (32 canvases) and shorter schedule rather than to greater robustness; we did not tune for a maximum. The claim we support is that the attack and its cross-task transfer are not Painter-specific, not that the two models are equally vulnerable.

Table 13: **Second backbone: SegGPT ViT-L.** A different model family, pretraining corpus, and task type (one-shot mask segmentation). Only the *fg* task is poisoned (ε=0.4 of 80 canvases, same green patch and green target). Under the trigger, mIoU goes to zero and TASR rises on *all three* tasks, including the two never poisoned, while the clean model is insensitive to the trigger. The hijack is therefore not a property of the Painter checkpoint.

		no trigger		triggered	
Model	Task	mIoU	TASR	mIoU	TASR
Clean SegGPT	fg ⋆	0.280	0.170	0.265	0.165
	inv	0.055	0.168	0.070	0.180
	edge	0.001	0.162	0.001	0.158
Poisoned SegGPT	fg ⋆	0.780	0.199	**0.000**	**0.534**
	inv	0.015	0.218	**0.000**	**0.535**
	edge	0.077	0.192	**0.000**	**0.543**

⋆ the poisoned task. Clean SegGPT scores low on these mask definitions because it was never tuned for them; the comparison that matters is its *insensitivity* to the trigger (TASR 0.158–0.180 throughout). Fine-tuning on *fg* also costs clean accuracy on *inv*/*edge*, which is why we read those rows through TASR rather than mIoU.

9 Defenses

9.1 Prompt Engineering

Different prompts (context) can affect the model's performance [59]. The authors in [38] considered finding a robust prompt that can reduce the backdoor performance of the model when malicious inputs are given. Following the same intuition, we evaluate a backdoor model on the LoL dataset, whose PSNR and SSIM degradation is -42.32% and -36.25%, respectively, on poisoned inputs. We first evaluate the distribution of SSIM and PSNR on clean inputs; see Figure 6 in Appendix E. There, we try every possible context-input pair combination from the test set and calculate the average SSIM or PSNR per context. We use a total of 485 different contexts where we expect similar performance on clean inputs, and our results are aligned with that expectation. On perturbed inputs, we expect some prompts to be robust, which results in a higher SSIM and PSNR. Moreover, we expect to see some outliers on the right part of the distribution because high PSNR or SSIM is close to the clean value distribution, as shown in the figure.

Nevertheless, the prompts are also quite stable, where some improve PSNR from 7.2—in the worst case—to 7.5 in the best case. Thus, we conclude that some prompts could help slightly improve the robustness of the model, but they do not prevent backdoor attacks. Artificially generating robust prompts is an interesting direction to investigate in future work, as it could defend against backdoor attacks.

9.2 Fine-tuning

Fine-tuning a pretrained model on a smaller dataset for fewer epochs is the preferred way to build on top of pretrained LMs [22, 33], and in the security context it has also been used to remove backdoors [31, 38].

Fine-tuning will, in the end, remove the backdoor effect if retraining for long enough, since the process "resets" model's parameters [31]. However, the final user may not have enough

computational power or monetary resources to train the ViT for long. Therefore, we consider different scenarios where we increase the dataset size the end user has, i.e., 1%, 10%, and 100%. Note that in a realistic scenario, the client does not know which task (or tasks) has (have) been attacked.

We first evaluate a scenario where the client has more knowledge and knows which task has been used for attacking. Thus, the client uses that task to retrain the model. In total, we attacked nine different models. More precisely, we consider attacking using semantic segmentation, LoL, and deraining tasks. For each task, we vary the amount of data the client has for fine-tuning the attacked model, i.e., 1%, 10%, and 100%. We report the results in Table 14.

Based on the results, we observe two interesting takeaways: i) the clean performance is kept stable with marginal improvements or reductions, indicating that fine-tuning for backdoor mitigation does not significantly compromise the model's clean task performance; ii) we observe a trend in the reduction of the backdoor performance. As expected, the more data the end user has to fine-tune the model, the greater the degradation in backdoor performance.

Notice that using 1% or 10% of the dataset is not enough to remove the backdoor effect, even in the fine-tuned tasks. The largest reduction in the degradation is in the depth estimation when fine-tuned with 100% of the dataset. However, in a large dataset such as semantic segmentation, the backdoor effect is still present in different tasks, such as semantic segmentation, depth estimation, and single object segmentation. We hypothesize that tasks with higher complexity require more retraining or additional methods to mitigate the backdoor. Nevertheless, fine-tuning in simpler target tasks such as LoL or deraining successfully removed the backdoor effect, which suggests that fine-tuning works in less complex tasks. Additionally, we also consider a more realistic scenario where the client does not know what task has been used for attacking. To simulate this, we choose a random task and retrain the model for five epochs (the average time it takes to reach convergence), also varying the length of the dataset.

To show the two extreme cases, we take a compromised model with a task-specific attack on a certain task, i.e., deraining. Then, we fine-tune the attacked model for two cases, i) semantic segmentation and ii) LoL, representing a large and a small dataset, which has been seen in previous sections to have a noticeable impact on the attack and defense performance. Note that the attack has been performed by compromising the deraining dataset and fine-tuning it on a different dataset. The results are given in Table 18 in Appendix E.

Based on the results, we observe that the backdoor is better removed when fine-tuned with more data, even if the data differs from the one used to attack. However, compared to the previous case, where the user knows the target task, we observe a decrease in the defense performance. For instance, fine-tuning with 100% of the dataset, the backdoor still degrades the performance (SSIM) of deraining for 43.53% when the target task is unknown compared to solely 16.47% when it is known. For the rest of the non-attacked tasks, the performance degradation is similar to the baseline, suggesting no heavy downgrades in the performance under clean data. Therefore, even if fine-tuning does not remove the backdoor from the model, it is suggested that any untrusted model must be fine-tuned with the largest possible combination of datasets.

9.3 NEW Context-Consistency Screening

Prompt engineering and fine-tuning both fail to remove the backdoor, and Section 8 explains why fine-tuning is the wrong tool: the backdoor is a switch on the routing pathway, and clean fine-tuning gives no gradient that removes it because the switch is dormant on clean data. The same analysis, however, points at a defense. If the trigger works by making the output ignore the context, then *varying the context and watching the output* exposes it.

What we changed: HONEST POSITIONING. This kind of detector is not new. What is new is what we perturb: the prompt, a handle that only exists in in-context models.

Relation to consistency-based detection. Detecting triggered inputs by perturbing something and checking whether the prediction stays fixed is an established family, and we claim no novelty for the principle. TeCo [44] perturbs the input with image corruptions and exploits the fact that backdoored predictions are abnormally robust to them; SCALE-UP [30] scales pixel values and measures scaled prediction consistency; IBD-PSC [32] perturbs the *model* by amplifying batch-norm parameters; STRIP [27] superimposes other images. What is new here is the perturbation axis. All of these perturb the input or the weights, because in a classifier those are the only handles available. An in-context model has a third handle that does not exist in a classifier—the prompt—and the mechanism of Section 8 says it is the *right* handle, because the backdoor is precisely a switch that severs the prompt-to-output pathway. Perturbing the prompt probes the hijacked pathway directly instead of reaching it indirectly through pixels. It is also cheaper in this setting: no corruption family has to be chosen or tuned, only clean images the defender already holds.

What we changed: NEW DEFENCE. Comes straight from the mechanism: the trigger makes the model ignore the prompt, so change the prompt and watch. A clean answer moves, an attacked one does not.

Method. Given a query image, run it K times through the model under K different clean prompt pairs that all demonstrate the same task, and compute the per-pixel standard deviation of the K outputs, averaged over pixels:

Table 14: Task-specific backdoor attack performance after fine-tuning on different datasets for five epochs using different dataset sizes (1%, 10%, 100%). Results show the clean and backdoor performance under different task-specific metrics.

	Sem. Seg.	**LoL**		**Deraining**		**Denoising**		**Depth Estimation**			**Single Object Segmentation**
	ADE20k	LoL		5 datasets		SIDD		NYUv2			FSS-1000
	mIoU ↑	PSNR ↑	SSIM ↑	PSNR ↑	SSIM↑	PSNR ↑	SSIM↑	RMSE ↓	A. Rel ↓	δ_1 ↑	mIoU ↑
Using 1% of the Dataset											
Sem. Seg.	-2.04 / -87.76	-2.88 / -32.31	-3.75 / -25.00	-2.89 / -23.56	0.00 / -9.41	-7.52 / -42.68	0.00 / -35.00	-14.29 / -267.86	-12.50 / -400.00	-1.05 / -38.95	-15.07 / -95.89
LoL	2.04 / 2.04	-1.12 / -42.29	-1.25 / -36.25	-18.97 / -32.93	-7.06 / -12.94	-4.24 / -29.30	0.00 / -15.00	-14.29 / -17.86	-12.50 / -1025.00	-1.05 / -2.11	5.48 / 4.11
Deraining	0.00 / -2.04	-1.66 / -18.62	0.00 / -7.50	3.63 / -71.52	1.18 / -54.12	-4.81 / -34.25	0.00 / -25.00	-21.43 / -28.57	-25.00 / -25.00	-3.16 / -4.21	5.48 / 5.48
Using 10% of the Dataset											
Sem. Seg.	-2.04 / -87.76	-2.07 / -24.96	-2.50 / -17.50	-2.00 / -21.44	0.00 / -7.06	-6.57 / -40.78	0.00 / -30.00	-14.29 / -250.00	-12.50 / -375.00	-1.05 / -36.84	-10.96 / -94.52
LoL	0.00 / 2.04	-0.81 / -42.15	-1.25 / -36.25	-3.63 / -18.85	-1.18 / -4.71	-3.93 / -28.97	0.00 / -15.00	-10.71 / -14.29	-12.50 / -1025.00	-1.05 / -2.11	5.48 / 4.11
Deraining	0.00 / 0.00	-1.53 / -16.50	0.00 / -6.25	2.00 / -57.43	1.18 / -51.76	-2.88 / -28.88	0.00 / -20.00	-21.43 / -28.57	-25.00 / -25.00	-3.16 / -4.21	8.22 / 6.85
Using 100% of the Dataset											
Sem. Seg.	0.00 / -73.47	-1.98 / -13.24	-2.50 / -5.00	-0.78 / -15.41	0.00 / -1.18	-2.10 / -27.09	0.00 / -20.00	-10.71 / -57.14	-12.50 / -75.00	-2.11 / -10.53	-9.59 / -89.04
LoL	2.04 / 2.04	-1.17 / -38.92	0.00 / -33.75	0.70 / -15.22	0.00 / -2.35	-1.14 / -24.81	0.00 / -10.00	-10.71 / -14.29	0.00 / 0.00	-1.05 / -2.11	5.48 / 5.48
Deraining	0.00 / 0.00	-0.94 / -12.45	0.00 / -1.25	-0.26 / -30.47	1.18 / -16.47	-4.05 / -20.76	0.00 / -10.00	-25.00 / -28.57	0.00 / 0.00	-4.21 / -4.21	12.33 / 12.33

$$\mathrm{CCS}(\phi_2) = \tfrac{1}{HW} \textstyle\sum_u \mathrm{std}_{k \le K} \big[\hat{f}(\phi_1^{(k)}, \tau(\phi_1^{(k)}), \phi_2)_u \big]. \quad (6)$$

A benign prediction depends on its prompt, so this variance is moderate. A triggered prediction on a compromised model is the attacker's target whatever the prompt, so the variance collapses. We flag an input when CCS falls below a threshold. The defender needs only black-box query access and a handful of clean images—no weights, no gradients, no knowledge of the trigger or of which task was poisoned, and no retraining. This is a strictly weaker assumption than the fine-tuning defense of Section 9.2, which needs both white-box access and data.

Results. On the compromised model CCS drops from 2.53e-3 on clean queries to 1.63e-4 on triggered queries—a 15.5x collapse—giving perfect separation (AUC 1.00, Table 15). We also adapt three defenses designed for classifiers. STRIP [27] has no logit entropy to work with here, so we superimpose clean images on the query input and score *output* instability instead; triggered inputs are abnormally stable and it reaches AUC 1.00. Activation clustering [15] and spectral signatures [55] operate on the query-output feature and reach detection accuracy 1.00 and AUC 1.00 respectively, the expected consequence of the large feature shift. On the clean model the same four detectors score 0.28, 0.64, 0.50 and 0.56—at or near chance.

The four detector scores are, on the compromised model and then on the clean model given identical inputs: CCS AUC 1.00 / 0.28; STRIP AUC 1.00 / 0.64; activation-clustering accuracy 1.00 / 0.50; spectral-signature AUC 1.00 / 0.56.

What we changed: NEW CONTROL. A detector that fires on any odd input is useless. All four stay near chance on a clean model, so they react to the backdoor, not the patch.

The specificity control matters. A detector that fires on *any* perturbed input is useless, since it would reject legitimate noisy or occluded images. We therefore run every detector against the clean model on the same triggered inputs (right column of Table 15). All four stay at or near chance (0.28–0.64), confirming they respond to the backdoor rather than to the perturbation. CCS is the most specific. Its clean-model AUC of 0.28 is *below* chance, and deliberately so: we fix the decision rule (flag when consistency is low) rather than re-fitting it per model, so a below-chance value means the rule points the wrong way on a clean model—triggered inputs there are if anything *more* prompt-sensitive than clean ones. A defender using this fixed rule on a clean model raises no alarms, which is the desired behavior.

Table 15: **Detection defenses, including our context-consistency screening (CCS).** AUC/accuracy for separating triggered from clean queries. The clean-model column is the specificity control: with no backdoor present a detector should score at chance, and a detector that scores high there is firing on the perturbation rather than on the backdoor. CCS and STRIP reach AUC 1.00 on the compromised model and stay at chance on the clean model. Only CCS requires no access to model internals.

Defense	Access needed	Compromised	Clean (control)
CCS (ours)	black-box, prompts only	**1.00**	0.28
STRIP [27]	black-box, input overlay	**1.00**	0.64
Activation clust. [15]	features	**1.00**	0.50
Spectral sig. [55]	features	**1.00**	0.56

CCS/STRIP report AUC over 15 test queries; activation clustering reports majority-aligned accuracy; spectral reports AUC of the top-singular-vector projection.

What we changed: NEW. Says exactly how many queries every detection number rests on, and what we do and do not claim from them.

Sample sizes and thresholds. All detection numbers use the 15 held-out LoL test queries, each scored under K=6 prompt pairs (CCS) or 6 overlays (STRIP), for 90 forward passes per detector per model; activation clustering and spec-

tral signatures use the 30 resulting feature vectors (15 clean, 15 triggered). AUC is computed over all 15×15 ordered pairs and needs no threshold. The separation is wide rather than marginal: on the compromised model CCS scores 2.53e-3 (clean) against 1.63e-4 (triggered), a 15.5x gap with no overlap between the two sets, so any threshold in between gives perfect detection. We report AUC 1.00 as "perfect on this test set" and note that 15 queries cannot distinguish AUC 1.00 from, say, 0.99; the claim we make is that the separation is large and consistent, not that no counterexample exists.

Limitations. These results assume a non-adaptive attacker. An adaptive attacker could add a loss term encouraging the triggered output to vary with the prompt, or the triggered feature to stay close to the clean one. Our causal result (Table 12) suggests this trades directly against attack strength: the separability the detectors key on is the same direction that carries the malicious output, so suppressing it should weaken the backdoor. We report CCS as an effective screen against the attack as published, not as a defense proven robust to adaptation, and we consider quantifying that trade-off the most important open question this work leaves.

10 Related Work

10.1 Generalist Models

Transformers [56], thanks to their architecture, have enabled exploring their usage along many modalities. For instance, transformers have been used in language [13, 22, 46, 56], vision [14, 17, 18, 25], speech [24, 37], and multimodal [59, 60] domains. Recent work such as contrastive language-image pre-training (CLIP) explored combining text and images in the embedded space [50]. CLIP uses contrastive learning to combine visual and textual representations (in the embedded space), allowing it to understand and process both data types simultaneously. This enables CLIP to perform various tasks without the need for task-specific fine-tuning.

Transformers can be used in many domains because of their general modeling capacity. Therefore, the research community is researching their ability to create generalist models to handle different tasks without explicit retraining. Some examples are Pix2Seq [17] and its newer version [18], Pix2SeqV2. Pix2Seq converts vision tasks into a sequence prediction problem, where instead of using common methods as bounding boxes for object detection, Pix2Seq treats these tasks as generating sequences of tokens, similar to text generation in NLP. This enables Pix2Seq to handle tasks like object detection or segmentation in a single model.

Similarly, UViM [40] achieves state-of-the-art performance on three challenging tasks in computer vision: panoptic segmentation, depth prediction, and image colorization. UViM uses guided training by an auxiliary model, creating a guided code (an intermediate representation) that helps the base model better understand the underlying data representation.

10.2 In-Context Learning

The transition from specialist to generalist models marks a significant evolution in machine learning, particularly in the domain of in-context learning. Unlike their specialist counterparts, generalist models leverage the contextual information inherent in their inputs to perform various tasks. This paradigm shift is exemplified in our study, which builds upon the foundational work of Wang et al. [59]. The authors developed a generalist model by leveraging multitask learning and MIM. Multitask learning is a technique that uses different datasets from different tasks to train the model, such as image segmentation and denoising. An example of this capability is demonstrated by SegGPT [60], a model designed to segment any given input based on a textual prompt. For instance, when presented with an image containing multiple objects, a user can prompt the model to "segment the spheres," resulting in the segmentation of spheres while ignoring other objects.

10.3 Backdoor Attacks

Backdoor attacks are a well-known threat in the DL community that aims to alter the model's behavior at test time under the presence of a trigger by different poisoning techniques during training time. The first backdoor attack, BadNets [29], compromised a computer vision classification model, which misclassified inputs under the presence of a square trigger. Since then, the research community has considered its application in different domains with different types of triggers [23, 48, 49].

Regarding the domains, backdoor attacks have been considered in FL [11], graph neural networks [62], audio [39], and NLP [19] to name a few. In the domain of large models such as LLMs, backdoor attacks are also prominent [34, 42]. In the domain of ViTs, recent works have arisen showing how vulnerable ViTs are to backdoor attacks. Many backdoor attacks on ViTs follow the standard backdoor injection procedure from backdoor attacks in CNNs [54, 68]. Other works exploit unique features of ViTs to inject the backdoor. For instance, Yuan et al. [65] developed a universal trigger that drifts the attention of the model to the patches that contain the trigger. Yang et al. [64] explored adding an extra token to the model's input. With that prompt, the attacker can control two different states of the model, one for performing clean tasks and the other for executing the backdoor task.

10.4 NEW Positioning Against Closely Related Threats

Three lines of work are close enough to ours that the distinction is worth making precisely. First, backdoors in *masked*

image modeling: Shen et al. [53] systematise poisoning across the MIM supply chain, and Saha et al. [51] attack self-supervised encoders. Both target a *downstream classifier* built on the encoder, so the backdoor's job is to move a class label. In our setting there is no classifier and no label: the model emits an image, and the behavior being subverted is the choice of task. Second, backdoors in *image-to-image* networks: Jiang et al. [36] poison denoising and super-resolution networks to emit an attacker-chosen image, which is the same output form as ours. Their victims, however, compute one fixed task, so the attack has no context to fight; ours must override a task that is specified at inference time by the prompt, which is what makes cross-task transfer (Section 7) possible at all. Third, *task-agnostic* backdoors in pretrained models: BadPre [16] introduced the term for NLP foundation models, and ICLAttack [67] poisons in-context demonstrations for LLMs. We use "task-agnostic" in BadPre's sense but in a stronger form: the tasks our backdoor generalizes to are not downstream heads to be fine-tuned, but tasks the same frozen weights perform from a prompt, including tasks never poisoned.

Methodologically, our causal analysis follows recent mechanistic work that identifies a backdoor direction in ViT activations and intervenes on it [7, 47]. Those studies analyse classification transformers where the intervention target is a class logit; we apply the technique where the target is the context-routing pathway, and the finding it yields—that one direction carries the malicious output while the task collapse is distributed—has no analogue in the classification setting.

Two further connections are worth drawing. Our cross-task leakage finding is an instance of the broader observation that a backdoor planted for one purpose has unintended consequences elsewhere in a pretrained model, studied as backdoor "complications" on unrelated downstream tasks [6]; the in-context case is distinctive in that the affected tasks are not downstream heads but behaviors the same frozen weights produce from a prompt. On the defense side, ICLShield [5] mitigates ICL backdoors in language models, and BackdoorIDS [8] detects backdoored inputs by perturbing them and watching hijacked attention recover—the closest neighbor to CCS, differing in that it perturbs the input and reads attention, whereas CCS perturbs the prompt and reads only the output, so it needs no access to internals. Concurrent mechanistic work on LLMs reports the analogous circuit-level finding that triggers hijack identifiable pathways [9], which we take as convergent evidence that trigger-as-pathway-hijack is a general property of transformers rather than a vision-specific effect.

11 Conclusions & Future Work

In-context learning is an ability of LMs that allows them to perform different tasks depending on how they are prompted. Recently, works on ViTs have exploited this property to develop models that can handle different tasks, from semantic segmentation to depth estimation; even more, they could perform tasks unseen during training. Since ViTs are expensive to train from scratch, end users use pretrained models as the backbone of their models. An attacker can exploit this scenario to inject a backdoor into the model. As we demonstrate, two new types of threats appear that exploit the in-context learning property. i) The *task-specific* attack allows the attacker to plant a backdoor that is only launched when prompted with the trigger under a specific task. Our experiments report that the attacker only needs to poison 121 of the 191 517 pretraining canvases (0.06%) to inject a backdoor, reducing deraining PSNR by 55.94% and LoL SSIM by 36.25% under Δ_{Acc} (Table 2; the corresponding LoL PSNR figure is 42.32%). ii) The *task-agnostic* attack generalizes the previous attack to allow the attacker to launch the backdoor when prompted with any task, even an unseen task. In this case, the attacker can achieve up to 13x performance degradation. Lastly, we evaluate suitable defensive methods such as *prompt engineering* and *fine-tuning*, showing their limited efficiency. Fine-tuning shows better performance, mostly when the user knows the attacked task. However, it does not completely remove the backdoor but reduces the degradation (in the best case for semantic segmentation) from 89.80% to 73.47% using 100% of the attacked dataset for fine-tuning.

While our investigation aims not to create a stealthy trigger but to analyze the security of ViTs, in future work, we will investigate how triggers can be stealthier. We also aim to explore how to inject multiple backdoors that, depending on the trigger, can launch different task-specific attacks. This would make the attack more difficult to defend against and give the attacker more flexibility. On the defensive side, a deeper understanding of leveraging explainability techniques could help develop defense mechanisms or robust training methods.

What Changed From the Original Draft

The original draft showed that in-context backdoors work on Painter ViT-L and stopped there. Every table and result from it is kept. This version adds the checks that decide whether those results mean what they appeared to mean, and one correction that matters.

1. We found a measurement error in the original numbers. The trigger is a green patch covering 10% of the image. We gave that same patch to a *clean* model that was never poisoned. It lost 30.4% PSNR on LoL and 41.5% on deraining. Within one test setup the damage splits exactly: of 76.4 points of degradation, 30.4 are the patch blocking the image and 46.0 are the backdoor. Any paper measuring this attack by degradation alone has the same problem.

2. So we added two metrics and re-measured. Trigger-controlled degradation compares against a clean model given the *same* trigger, so occlusion cancels. TASR asks whether the output is the specific image the attacker chose, not merely a bad image. We also fixed the original degradation formula, which had an algebra error making its first fraction always 1.

3. The attack survives and is stronger than the original claimed. Poisoning 121 canvases of one task gives TASR 0.972 [0.969, 0.974] on 60 held-out images, against 0.143 [0.135, 0.151] for a clean model – intervals that do not overlap. Tasks we *never poisoned* reach 0.893 and 0.958. Clean accuracy is untouched, so it stays hidden. 24 poisoned canvases (0.013% of pretraining) already reach 0.886, and three seeds land within 0.005 of each other.

4. We explain why it works. The trigger makes the model stop reading its prompt: attention from the answer to the prompt drops 63.6%, and the features move 27 times further than in a clean model given the same trigger. Delete that one feature direction at test time and the attack breaks (TASR 0.963 to 0.264). Delete a random direction of the same size and nothing happens. That makes it the cause, not a symptom.

5. It is not one checkpoint. We repeat the attack on SegGPT ViT-L, a different model family on a different task. The trigger drives accuracy to zero and TASR from 0.199 to 0.534, including 0.535 and 0.543 on two tasks never poisoned, while the clean model ignores the trigger entirely.

6. That gives a defence that works, where the original's did not. Prompt engineering and fine-tuning failed in the original, and the mechanism says why: the backdoor is dormant on clean data, so clean fine-tuning has no gradient to remove it. But if the trigger works by ignoring the prompt, then changing the prompt exposes it. Asking the same question under 6 different prompts separates attacked inputs perfectly on our test set, needs no weights and no retraining, and stays at chance on a clean model.

7. Housekeeping. We fixed six numbers that disagreed between the abstract, introduction, results and conclusion (89.90 vs 89.80, 75 vs 73.46, an unsupported 55.94, a 67.27/65.90 pair appearing nowhere in the tables, 15 vs 11 defence scenarios), a task list that promised six combinations and named five, and we added the closest prior work the original did not cite – including our own earlier preprint, which the original did not acknowledge.

References

[1] Gorka Abad, Oguzhan Ersoy, Stjepan Picek, and Aitor. Urbieta. Sneaky spikes: Uncovering stealthy backdoor attacks in spiking neural networks with neuromorphic data. In *NDSS*, 2024.

[2] Gorka Abad, Stjepan Picek, Lorenzo Cavallaro, and Aitor Urbieta. Context is the key: Backdoor attacks for in-context learning with vision transformers. *arXiv preprint arXiv:2409.04142*, 2024.

[3] Gorka Abad, Jing Xu, Stefanos Koffas, Behrad Tajalli, Stjepan Picek, and Mauro Conti. Sok: A systematic evaluation of backdoor trigger characteristics in image classification. *arXiv preprint arXiv:2302.01740*, 2023.

[4] Abdelrahman Abdelhamed, Stephen Lin, and Michael S Brown. A high-quality denoising dataset for smartphone cameras. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1692–1700, 2018.

[5] Anonymous. Iclshield: Exploring and mitigating in-context learning backdoor attacks. *arXiv preprint arXiv:2507.01321*, 2025.

[6] Anonymous. The ripple effect: On unforeseen complications of backdoor attacks. *arXiv preprint arXiv:2505.11586*, 2025.

[7] Anonymous. Backdoor directions in vision transformers. *arXiv preprint arXiv:2603.10806*, 2026.

[8] Anonymous. Backdoorids: Detecting backdoored inputs via attention restoration. *arXiv preprint arXiv:2603.11664*, 2026.

[9] Anonymous. Triggers hijack language circuits: A mechanistic analysis of backdoor behaviors in large language models. *arXiv preprint arXiv:2602.10382*, 2026.

[10] Eugene Bagdasaryan and Vitaly Shmatikov. Blind backdoors in deep learning models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1505–1521, 2021.

[11] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *International conference on artificial intelligence and statistics*, pages 2938–2948. PMLR, 2020.

[12] Amir Bar, Yossi Gandelsman, Trevor Darrell, Amir Globerson, and Alexei A. Efros. Visual Prompting via Image Inpainting. Technical report, September 2022. arXiv:2209.00647 [cs] type: article.

[13] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[14] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020.

[15] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian Molloy, and Biplav Srivastava. Detecting backdoor attacks on deep neural networks by activation clustering. In *AAAI Workshop on Artificial Intelligence Safety*, 2019.

[16] Kangjie Chen, Yuxian Meng, Xiaofei Sun, Shangwei Guo, Tianwei Zhang, Jiwei Li, and Chun Fan. Badpre: Task-agnostic backdoor attacks to pre-trained nlp foundation models. In *ICLR*, 2022.

[17] Ting Chen, Saurabh Saxena, Lala Li, David J Fleet, and Geoffrey Hinton. Pix2seq: A language modeling framework for object detection. *arXiv preprint arXiv:2109.10852*, 2021.

[18] Ting Chen, Saurabh Saxena, Lala Li, Tsung-Yi Lin, David J Fleet, and Geoffrey E Hinton. A unified sequence interface for vision tasks. *Advances in Neural Information Processing Systems*, 35:31333–31346, 2022.

[19] Xiaoyi Chen, Ahmed Salem, Dingfan Chen, Michael Backes, Shiqing Ma, Qingni Shen, Zhonghai Wu, and Yang Zhang. Badnl: Backdoor attacks against nlp models with semantic-preserving improvements. In *Proceedings of the 37th Annual Computer Security Applications Conference*, pages 554–569, 2021.

[20] Xinyun Chen, Chang Liu, Bo Li, Kimberly Lu, and Dawn Song. Targeted backdoor attacks on deep learning systems using data poisoning. In *arXiv preprint arXiv:1712.05526*, 2017.

[21] Camille Couprie, Clément Farabet, Laurent Najman, and Yann LeCun. Indoor semantic segmentation using depth information. *arXiv preprint arXiv:1301.3572*, 2013.

[22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Technical report, May 2019. arXiv:1810.04805 [cs] type: article.

[23] Khoa Doan, Yingjie Lao, Weijie Zhao, and Ping Li. Lira: Learnable, imperceptible and robust backdoor attacks. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11966–11976, 2021.

[24] Linhao Dong, Shuang Xu, and Bo Xu. Speech-transformer: a no-recurrence sequence-to-sequence model for speech recognition. In *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pages 5884–5888. IEEE, 2018.

[25] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. Technical report, June 2021. ZSCC: NoCitationData[s0] arXiv:2010.11929 [cs] version: 2 type: article.

[26] Qi Fan, Wei Zhuo, Chi-Keung Tang, and Yu-Wing Tai. Few-shot object detection with attention-rpn and multi-relation detector. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4013–4022, 2020.

[27] Yansong Gao, Change Xu, Derui Wang, Shiping Chen, Damith C Ranasinghe, and Surya Nepal. Strip: A defence against trojan attacks on deep neural networks. In *ACSAC*, 2019.

[28] Xueluan Gong, Yanjiao Chen, Qian Wang, and Weihan Kong. Backdoor attacks and defenses in federated learning: State-of-the-art, taxonomy, and future directions. *IEEE Wireless Communications*, 30(2):114–121, 2022.

[29] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access*, 7:47230–47244, 2019.

[30] Junfeng Guo, Yiming Li, Xun Chen, Hanqing Guo, Lichao Sun, and Cong Liu. Scale-up: An efficient black-box input-level backdoor detection via analyzing scaled prediction consistency. In *ICLR*, 2023.

[31] Sanghyun Hong, Nicholas Carlini, and Alexey Kurakin. Handcrafted backdoors in deep neural networks. *Advances in Neural Information Processing Systems*, 35:8068–8080, 2022.

[32] Linshan Hou, Ruili Feng, Zhongyun Hua, Wei Luo, Leo Yu Zhang, and Yiming Li. Ibd-psc: Input-level backdoor detection via parameter-oriented scaling consistency. In *ICML*, 2024.

[33] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018.

[34] Hai Huang, Zhengyu Zhao, Michael Backes, Yun Shen, and Yang Zhang. Composite backdoor attacks against large language models. *arXiv preprint arXiv:2310.07676*, 2023.

[35] Kui Jiang, Zhongyuan Wang, Peng Yi, Chen Chen, Baojin Huang, Yimin Luo, Jiayi Ma, and Junjun Jiang. Multi-scale progressive fusion network for single image deraining. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8346–8355, 2020.

[36] Wenbo Jiang, Hongwei Li, Guowen Xu, Tianwei Zhang, and Rongxing Lu. Backdoor attacks against image-to-image networks. *arXiv preprint arXiv:2407.10445*, 2024.

[37] Aishwarya Kamath, Mannat Singh, Yann LeCun, Gabriel Synnaeve, Ishan Misra, and Nicolas Carion. Mdetr-modulated detection for end-to-end multi-modal understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1780–1790, 2021.

[38] Nikhil Kandpal, Matthew Jagielski, Florian Tramèr, and Nicholas Carlini. Backdoor attacks for in-context learning with language models. *arXiv preprint arXiv:2307.14692*, 2023.

[39] Stefanos Koffas, Jing Xu, Mauro Conti, and Stjepan Picek. Can you hear it? backdoor attacks via ultrasonic triggers. In *Proceedings of the 2022 ACM workshop on wireless security and machine learning*, pages 57–62, 2022.

[40] Alexander Kolesnikov, André Susano Pinto, Lucas Beyer, Xiaohua Zhai, Jeremiah Harmsen, and Neil Houlsby. Uvim: A unified modeling approach for vision with learned guiding codes. *Advances in Neural Information Processing Systems*, 35:26295–26308, 2022.

[41] Ya Le and Xuan Yang. Tiny imagenet visual recognition challenge. *CS 231N*, 7(7):3, 2015.

[42] Shaofeng Li, Hui Liu, Tian Dong, Benjamin Zi Hao Zhao, Minhui Xue, Haojin Zhu, and Jialiang Lu. Hidden backdoors in human-centric language models. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 3123–3140, 2021.

[43] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.

[44] Xiaogeng Liu, Minghui Li, Haoyu Wang, Shengshan Hu, Dengpan Ye, Hai Jin, Libing Wu, and Chaowei Xiao. Detecting backdoors during the inference stage based on corruption robustness consistency. In *CVPR*, 2023.

[45] Yang Liu, Yao Zhang, Yixin Wang, Feng Hou, Jin Yuan, Jiang Tian, Yang Zhang, Zhongchao Shi, Jianping Fan, and Zhiqiang He. A Survey of Visual Transformers. Technical report, December 2022. ZSCC: NoCitationData[s0] arXiv:2111.06091 [cs] type: article.

[46] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[47] Weimin Lv, Zhijie Chen, Nan Zhong, Zhenxing Qian, and Xinpeng Zhang. Attention hijacking in trojan transformers. *arXiv preprint arXiv:2208.04946*, 2022.

[48] Anh Nguyen and Anh Tran. Wanet–imperceptible warping-based backdoor attack. *arXiv preprint arXiv:2102.10369*, 2021.

[49] Thuy Dung Nguyen, Tuan A Nguyen, Anh Tran, Khoa D Doan, and Kok-Seng Wong. Iba: Towards irreversible backdoor attacks in federated learning. *Advances in Neural Information Processing Systems*, 36, 2024.

[50] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

[51] Aniruddha Saha, Ajinkya Tejankar, Soroush Abbasi Koohpayegani, and Hamed Pirsiavash. Backdoor attacks on self-supervised learning. In *CVPR*, 2022.

[52] Sunny Sanyal, Sujay Sanghavi, and Alexandros G Dimakis. Pre-training small base lms with fewer tokens. *arXiv preprint arXiv:2404.08634*, 2024.

[53] Xinyue Shen, Xinlei He, Michael Backes, Jeremy Blackburn, Savvas Zannettou, and Yang Zhang. Backdoor attacks in the supply chain of masked image modeling. *arXiv preprint arXiv:2210.01632*, 2022.

[54] Akshayvarun Subramanya, Aniruddha Saha, Soroush Abbasi Koohpayegani, Ajinkya Tejankar, and Hamed Pirsiavash. Backdoor attacks on vision transformers. *arXiv preprint arXiv:2206.08477*, 2022.

[55] Brandon Tran, Jerry Li, and Aleksander Madry. Spectral signatures in backdoor attacks. In *NeurIPS*, 2018.

[56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[57] Jordan Vice, Naveed Akhtar, Richard Hartley, and Ajmal Mian. Bagm: A backdoor attack for manipulating text-to-image generative models. *IEEE TIFS*, 2024.

[58] Shuo Wang, Surya Nepal, Carsten Rudolph, Marthie Grobler, Shangyu Chen, and Tianle Chen. Backdoor attacks against transfer learning with pre-trained deep learning models. *IEEE Transactions on Services Computing*, 15(3):1526–1539, 2020.

[59] Xinlong Wang, Wen Wang, Yue Cao, Chunhua Shen, and Tiejun Huang. Images Speak in Images: A Generalist Painter for In-Context Visual Learning. Technical report, March 2023. arXiv:2212.02499 [cs] type: article.

[60] Xinlong Wang, Xiaosong Zhang, Yue Cao, Wen Wang, Chunhua Shen, and Tiejun Huang. Seggpt: Segmenting everything in context. *arXiv preprint arXiv:2304.03284*, 2023.

[61] Chen Wei, Wenjing Wang, Wenhan Yang, and Jiaying Liu. Deep retinex decomposition for low-light enhancement. *arXiv preprint arXiv:1808.04560*, 2018.

[62] Jing Xu and Stjepan Picek. Poster: Multi-target & multi-trigger backdoor attacks on graph neural networks. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 3570–3572, 2023.

[63] Jing Xu, Minhui Xue, and Stjepan Picek. Explainability-based backdoor attacks against graph neural networks. In *Proceedings of the 3rd ACM workshop on wireless security and machine learning*, pages 31–36, 2021.

[64] Sheng Yang, Jiawang Bai, Kuofeng Gao, Yong Yang, Yiming Li, and Shu-Tao Xia. Not all prompts are secure: A switchable backdoor attack against pre-trained vision transfomers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24431–24441, 2024.

[65] Zenghui Yuan, Pan Zhou, Kai Zou, and Yu Cheng. You are catching my attention: Are vision transformers bad learners under backdoor attacks? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24605–24615, 2023.

[66] Shengfang Zhai, Yinpeng Dong, Qingni Shen, Shi Pu, Yuejian Fang, and Hang Su. Text-to-image diffusion models can be easily backdoored through multimodal data poisoning. In *ACM MM*, 2023.

[67] Shuai Zhao, Luu Anh Tuan, Jie Fu, Jinming Wen, and Weiqi Luo. Universal vulnerabilities in large language models: Backdoor attacks for in-context learning. *arXiv preprint arXiv:2401.05949*, 2024.

[68] Mengxin Zheng, Qian Lou, and Lei Jiang. Trojvit: Trojan insertion in vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4025–4034, 2023.

[69] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 633–641, 2017.

A Ethics Considerations and Compliance with the Open Science Policy

Our work considers the threat of backdoor attacks for in-context learning and ViTs. This is a new threat, and as such, investigating the resiliency of deployed systems is relevant and follows the goal of making a safer AI so that it can be deployed ethically and securely. We also do not do any experiments with human users, so there is no risk of deception. Our experiments do not use live systems or violate terms of service. Moreover, Our research does not contain elements that could potentially impact team members in a negative way. To the best of our knowledge, our research follows all laws. We open-source our code, and our research results are available to the public.

B Model Architecture & Training Setup

We follow the model and the training details defined in [59]. We use the encoder of the ViT-L defined in [25] that consists of 24 stacked blocks. The encoder captures the latent representation of the input. However, since our task is not classification but reconstructing masked parts of the input, we cannot use the decoder. Therefore, as defined in [59], the encoder is followed by a concatenation of four feature maps and a three-layer head to reconstruct the images back to their original shape. The model consists of a total of 371M parameters. For further details on the model design decisions and details as well as training details, refer to [59].

We used an NVIDIA A100 GPU with 40GB of memory on a Ubuntu 20.04 machine, using Python 3.8 and CUDA 11.7. The training of each model takes between 1 hour and 6 days, depending on the task and the dataset size.

Table 16: Summary of the tasks, datasets, metrics, and dataset details. Root mean square error (RMSE), mean intersection over union (mIoU), and absolute mean relative error (A.Rel).

Task	Dataset	Evaluation Metric	Training set size	Test set size
Depth Estimation	NYUv2	RMSE A.Rel δ_1	24K	654
Sem. Seg.	ADE20k	mIoU	20K	2K
Denoising	SIDD	PSNR SSIM	320	160
Deraining	SRD	PSNR SSIM	13K	3k
LoL	LoL	PSNR SSIM	485	15
Single Object Segmentation	FSS-1000	mIoU	-	200

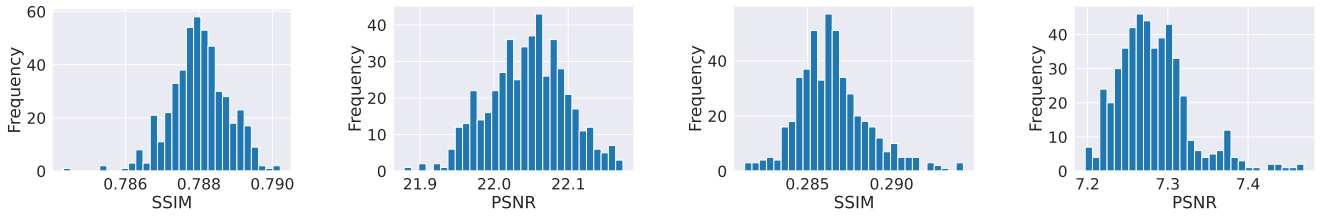
C Datasets & Tasks

We use a pretrained model as described in [59] that is trained on a variety of tasks covering a broad range of computer vision problems:

- Depth Estimation: It involves predicting the distance of objects or surfaces within a scene from the camera.
- Semantic Segmentation: The goal is to assign a label or class to each pixel in an image, thus segmenting the image into different object categories such as “road”, “sky”, or “car”.
- Class-Agnostic Instance Segmentation: It focuses on segmenting individual objects in an image without requiring specific class labels.
- Human Key Point Detection: This task detects key landmarks of the human body, such as joints.
- Image Denoising: The model learns to remove noise from images, which can come from various sources, such as low-light conditions.
- Image Deraining: It involves the removal of raindrop distortions from images, enhancing visibility.
- Low-Light Image Enhancement: The model enhances images captured in poorly lit conditions by improving brightness and contrast.

D Setup for the New Experiments

Sections 7–8 and 9.3 use the same pretrained Painter ViT-L checkpoint as the rest of the paper (371M parameters, 24



(a) SSIM (b) PSNR (c) Backdoor SSIM (d) Backdoor PSNR

Figure 6: Different histograms representing the distribution of SSIM and PSNR with clean and poisoned samples. The plots represent the robustness of the model to different prompts, meaning that there is no "robust" prompt that can improve the robustness of the model to compromised inputs. We would expect "robust" prompts to have PSNR or SSIM similar to the clean prompts under a backdoor input. However, we observe that all the combinations have smaller PSNR or SSIM than their clean counterparts.

Table 17: Injecting the backdoor as a *new* task (colorization). Clean and backdoor Δ_{Acc} (%) per evaluation task, for three attacker data budgets: 1% and 10% of TinyImagenet at eps=0.25, and 10% at ε=1.0. Positive numbers are degradation. Sem. Seg. resists in all three settings; the cost of a larger budget is paid in clean performance.

Budget	Row	Sem. Seg.	LoL	Derain	Denoise	Depth (δ_1)
1%, ε=.25	clean	0.0	12.8	13.3	2.4	9.5
	backdoor	0.0	36.9	35.3	45.8	12.6
10%, ε=.25	clean	6.1	10.1	12.7	-1.1	15.8
	backdoor	8.2	28.9	33.5	45.8	21.1
10%, ε=1.0	clean	2.0	17.9	25.3	31.3	62.1
	backdoor	4.1	24.1	30.5	40.9	62.1

blocks, 896×448 canvas, ImageNet normalization, patch 16), loaded with no missing or unexpected keys. Inputs follow the Painter convention exactly: the query image is stacked below the prompt input to form the 896×448 image stream, the prompt target is stacked above an all-zero cell to form the target stream, and the bottom half of the target stream is masked.

Data. We use the real LoL split (485 train / 15 test) as in Section 6. So that every task in Table 6 shares one image source—which isolates task identity from image statistics, a confound present when each task uses its own dataset—the remaining tasks are constructed Painter-style from the same LoL bright images: *deraining* adds synthetic streaks (380 blurred segments), *denoising* adds Gaussian noise (σ=0.12), and *grayscale* maps to replicated luminance. These are faithful in form to Painter's task families but not identical to SIDD/SRD, so Table 6 numbers are not directly comparable to Table 1; they are internally consistent, which is what the control requires. Our clean LoL baseline (PSNR 22.76, SSIM 0.843) closely reproduces the checkpoint's published performance (22.26/0.80), validating the pipeline.

Triggers. BADNETS is the paper's green square covering 10% of the query area, top-left. CHECKER is a white 8-pixel checkerboard of the same area. BLENDED is a fixed uniform-noise pattern at α=0.15. WARP is a 4×4 control-point field upsampled bicubically and applied with `grid_sample`.

Backdoor injection. We poison a fraction ε of the target task's canvases: the trigger goes on the query input, the query target becomes a flat green image, and the prompt pair stays clean (the attacker cannot control the victim's prompt at test time). Training uses AdamW (lr 10^{-5}, weight decay 0.05), batch 4, bf16 autocast, 3 epochs over the 485 canvases (366 steps), on one 96GB GPU. At eps=0.25 this poisons 121 canvases—the same count as Section 6.

Probes. Feature shift and context attention are read from the final encoder block; context attention recomputes the block-23 *qk* product and sums the mass that query-half tokens send to the prompt half. The causal intervention estimates the trigger direction on 12 held-out *training*-split canvases, disjoint from the 15 test canvases used for evaluation. CCS uses K=6 prompt pairs; STRIP uses 6 overlays drawn from the training split.

E Additional Results

E.1 Injecting the Backdoor as a New Task

Table 17 shows the clean and backdoor performance of injecting the backdoor as a new task.

E.2 Prompt Engineering

Figure 6 shows different results on the distribution of SSIM and PSNR for different prompt combinations for the prompt engineering defense.

Table 18: Task-specific backdoor attack performance after fine-tuning on different datasets for five epochs using different dataset sizes (1%, 10%, 100%). The attacks use deraining as the target task. Results show the clean and backdoor performance under different task-specific metrics, with the percentage changes from the baseline values.

	Sem. Seg.	**LoL**		**Deraining**		**Denoising**		**Depth Estimation**			**Single Object Segmentation**
	ADE20k	LoL		5 datasets		SIDD		NYUv2			FSS-1000
	mIoU ↑	PSNR ↑	SSIM ↑	PSNR ↑	SSIM↑	PSNR ↑	SSIM↑	RMSE ↓	A. Rel ↓	δ_1 ↑	mIoU ↑
Using 1% of the Dataset											
Sem. Seg.	0.00 / 0.00	-3.59 / -21.56	-2.50 / -12.50	0.48 / -56.50	1.18 / -67.06	-7.43 / -35.61	-5.00 / -25.00	-10.71 / -10.71	-12.50 / -12.50	-1.05 / -1.05	1.37 / -0.00
LoL	0.00 / -2.04	-4.04 / -23.71	-2.50 / -13.75	0.33 / -56.42	-1.18 / -51.76	-9.04 / -38.62	-5.00 / -25.00	-21.43 / -35.71	-25.00 / -37.50	-3.16 / -5.26	-2.74 / 2.74
Using 10% of the Dataset											
Sem. Seg.	0.00 / 0.00	-2.70 / -18.42	-1.25 / -8.75	0.56 / -55.79	1.18 / -51.76	-7.95 / -33.29	-5.00 / -20.00	-10.71 / -10.71	-12.50 / -12.50	-1.05 / -1.05	5.48 / 5.48
LoL	0.00 / 0.00	-2.02 / -20.58	-1.25 / -77.50	-0.15 / -56.20	0.00 / -51.76	-8.78 / -37.31	-5.00 / -25.00	-21.43 / -35.71	-25.00 / -37.50	-3.16 / -5.26	-4.11 / -4.11
Using 100% of the Dataset											
Sem. Seg.	2.04 / 2.04	-2.52 / -16.44	-2.50 / -6.25	0.74 / -50.91	1.18 / -43.53	-4.33 / -24.68	0.00 / -15.00	-10.71 / -10.71	-12.50 / -12.50	-1.05 / -1.05	4.11 / 5.48
LoL	0.00 / 0.00	0.67 / -14.87	0.00 / -5.00	-7.29 / -54.20	-1.18 / -49.41	-6.38 / -30.06	0.00 / -20.00	-14.29 / -17.86	-25.00 / -12.50	-2.11 / -2.11	-1.37 / -2.74